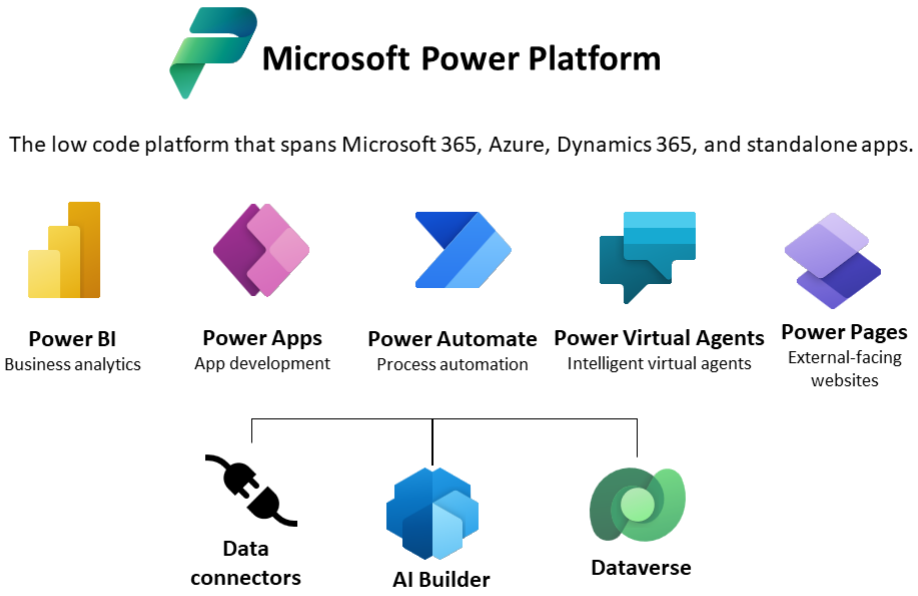
Dataverse, solutions, tables and model-driven apps

1. Microsoft Power Platform and Dataverse

The **Power Platform** is a suite of low-code and no-code tools provided by Microsoft for building custom business applications, automating workflows, and analyzing data. Currently it consists of five main components.



1. **Power BI**:

   - Purpose: Power BI is a business analytics tool used for visualizing and sharing insights from your organization's data.

   - Key Features:
     - Data visualization: create interactive reports and dashboards.
     - Data exploration: discover insights and trends in the data.
     - Integration: connect to various data sources, both on-premises and in the cloud.
     - Sharing: share reports and dashboards with others in your organization.

2. **Power Apps**:

   - Purpose: Power Apps is a platform for building custom applications with minimal coding.

   - Key Features:
     - Canvas Apps: build custom apps from scratch with a drag-and-drop interface.
     - Model-Driven Apps: create apps with a data-driven approach using a pre-defined data model.
     - Connectors: integrate with a wide range of data sources and services.
     - Dataverse/Common Data Service (CDS): store and manage data used by Power Apps applications.

3. **Power Automate**:

    - Purpose: Power Automate (formerly known as Microsoft Flow) allows users to automate workflows and processes across various applications and services.

    - Key Features:
        - Flow Templates: use pre-built templates for common workflows.
        - Connectors: integrate with a variety of apps and services.
        - Approvals: create automated approval processes.
        - Robotic Process Automation (RPA): automate repetitive tasks.

4. **Power Virtual Agents**:

    - Purpose: Power Virtual Agents enables the creation of chatbots with no or low code.

    - Key Features:
        - Chatbot Design: design chatbots using a visual interface.
        - Integration: connect chatbots to various data sources and services.
        - Natural Language Processing: use natural language understanding to build conversational bots.

5. **Power Pages**:

    - Purpose: Power Pages is a secure, enterprise-grade, low-code software as a service (SaaS) platform for creating, hosting, and administering modern external-facing business websites.

    - Key Features:
        - Simplified authoring experience for makers: quickly create new sites by using the default template or choosing from existing industry-based starter templates.
        - Design studio: build powerful and engaging sites without writing a single line of code.
        - Responsive rendering: Power Pages is based on Bootstrap, which natively provides support for building websites that are responsive, mobile-friendly, and available in various form factors.
        - Advanced development capabilities for pro developers: makers can work with pro developers in fusion teams to extend the functionality using Visual Studio Code and the Microsoft Power Platform CLI to create powerful business application websites.

Together, these components of the Power Platform empower users to solve business problems, streamline processes, and gain insights without the need for extensive coding skills. The Power Platform is designed to be used by both professional developers and business users, allowing organizations to create custom solutions that meet their unique requirements. Additionally, the components of the Power Platform work seamlessly together, enabling users to build end-to-end solutions that encompass data analysis, application development, automation, and more.

Power BI, Power Apps, Power Automate and Power Virtual Agents all require storage, which is **Dataverse**.

Here are the key aspects of Dataverse:

1. **Common Data Service (CDS)**: Dataverse was previously known as the Common Data Service (CDS). It serves as a scalable and secure data platform that allows organizations to securely store and manage data used by business applications. Dataverse provides a unified and standardized data model that applications built on the Power Platform can leverage.

2. **Standardized Data Model**: Dataverse includes a standard set of entities, fields, and relationships that cover common business scenarios. This standardized data model simplifies the process of building applications and ensures consistency across different solutions.

3. **Connectivity and Integration**: Dataverse allows seamless integration with other Microsoft 365 and Dynamics 365 services, as well as with external data sources. This integration capability is crucial for creating comprehensive and connected business solutions.

4. **Data Security and Permissions**: Dataverse includes robust security features, enabling organizations to control access to data. Security roles and permissions can be defined to restrict or grant access to specific entities and fields based on user roles.

5. **Automatic Updates and Maintenance**: Dataverse provides automatic updates and maintenance of the underlying infrastructure, allowing organizations to focus on building applications rather than managing the underlying data platform.

6. **Power Platform Integration**: Dataverse is tightly integrated with Power BI, Power Apps, Power Automate, and Power Virtual Agents. When we build applications using Power Apps, the data is often stored in Dataverse, providing a consistent and centralized data storage solution.

7. **Extensibility**: while Dataverse provides a standardized data model, it is also extensible. Organizations can add custom entities, fields, and business logic to meet specific requirements.

8. **Data Relationships**: Dataverse supports relationships between tables, allowing us to model complex business scenarios and ensure data consistency.

Summarizing, Dataverse serves to store and manage the data used by all applications. How is this data stored? Data is stored in the form of **tables**, which can be managed in a very simple and direct way. A table refers to a collection of data organized in rows and columns. Tables in Dataverse are like database tables, and they store records that represent instances of a particular entity. An **entity**, in this context, is a business concept or object that we want to model and manage data for. Entities are defined by tables in Dataverse.

Here are some key points about tables in Dataverse:

1. **Entities**:

    - An entity is represented by a table in Dataverse. Each table corresponds to a specific business entity, such as Account, Contact, or Product.

    - For example, if we are building a **CRM** (Customer Relationship Management) application, we might have tables/entities for Customer, Opportunity, and Case.

2. **Columns and Fields**:

    - Each table (entity) consists of **columns**, also known as **fields**. Columns define the attributes or properties of the entity.

    - Fields can represent different data types, such as text, numbers, dates, lookup fields, etc.

3. **Record**:

    - A **record** is an instance of an entity. It is a single row in the table that contains values for each field/column.

    - For example, a record in the Contact entity might represent an individual person and include fields such as First Name, Last Name, and Email.

4. **Primary Key**:

    - Each table in Dataverse has a **primary key**, which is a unique identifier for each record in the table. The primary key ensures the uniqueness of the records and is used to establish relationships between tables.

5. **Relationships**:

    - Tables in Dataverse can have **relationships** with other tables. Relationships define how records in one table relate to records in another table.

    - For example, there might be a relationship between the Account and Contact tables to associate contacts with specific customer accounts.

6. **Schema and Metadata**:

    - The structure of each table, including the definition of fields, relationships, and other metadata, is referred to as the **table's schema**.

    - Dataverse provides a consistent and standardized schema for tables, and it includes a set of predefined tables for common business scenarios.

7. **Security and Permissions**:

    - Tables in Dataverse are subject to **security roles** and **permissions**. Access to tables and their data can be controlled based on user roles.

2. The Sales module

In Dynamics 365, the Sales module is a key component of the platform specifically designed to help organizations manage their sales processes efficiently. Dynamics 365 for Sales provides a comprehensive set of tools and features to streamline sales activities, improve customer engagement, and drive sales success.

Here are some key aspects of the Sales module in Dynamics 365:

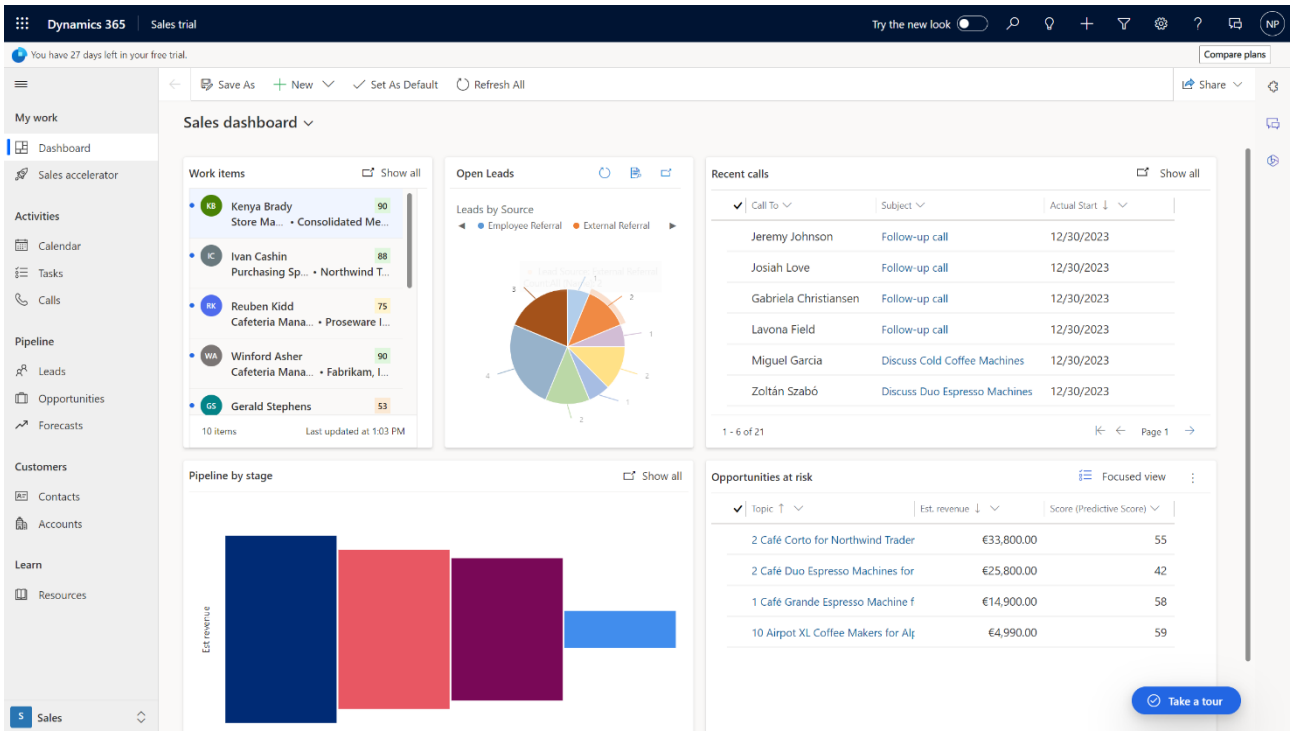
1. **Lead and Opportunity Management**:

    - Allows users to track and manage leads, qualify them into opportunities, and then progress through the sales pipeline.

    - Provides a unified view of customer interactions and history.

2. **Account and Contact Management**:
    - Enables the organization to maintain a central repository of customer accounts and contacts.
    - Helps in understanding customer relationships and communication history.
3. **Product and Price List Management**:
    - Supports the creation and management of product catalogs and price lists.
    - Allows sales representatives to associate products with opportunities and quotes.
4. **Opportunity and Quote Management**:
    - Facilitates the creation and tracking of sales opportunities.
    - Allows users to create quotes, send them to customers, and track the quoting process.
5. **Sales Process Automation**:
    - Provides tools for automating and standardizing sales processes.
    - Allows the creation of business process flows to guide sales reps through predefined stages.
6. **Pipeline Management and Forecasting**:
    - Enables sales managers to monitor the sales pipeline and forecast future sales based on opportunities.
    - Supports analytics and reporting on sales performance.
7. **Integration with Office 365**:
    - Integrates seamlessly with Microsoft Office 365 applications, such as Outlook, Excel, and SharePoint.
    - Allows users to work with sales data directly within familiar Office tools.
8. **Mobile Access**:
    - Offers a mobile app that allows sales representatives to access customer information, update records, and manage tasks while on the go.
9. **Customer Insights**:
    - Provides insights into customer behavior and preferences.
    - Helps in personalizing interactions and delivering a better customer experience.
10. **Integration with Power Platform**:
    - Integrates with other components of the Microsoft Power Platform, such as Power BI for analytics and Power Automate for workflow automation.

The Sales module in Dynamics 365 is designed to empower sales teams with the tools they need to build strong customer relationships, streamline sales processes, and make data-driven decisions. This is the view we have when we open the module in Dynamics 365 (first we must activate the Sales trial).



Leads, Opportunities, Contacts and Accounts all correspond to Dataverse tables.

Many data types are available in Dataverse, and we also have logic and validation tools. Logic and validation are implemented using various tools and features to ensure data integrity, enforce business rules, and automate processes. Here are some key tools and features related to logic and validation in Dataverse:

1. **Business Rules**:

    - Purpose: business rules in Dataverse allow us to define client-side logic to enforce business policies and logic without writing code.

    - Functionality:
        - Automatically set field values based on conditions.
        - Show or hide fields based on conditions.
        - Validate data entry on the client side.

2. **Workflows**:

    - Purpose: workflows are processes that can be automated to trigger based on certain conditions or events.

    - Functionality:
        - Set field values, create records, or send emails based on specified conditions.
        - Workflows are often used for automating routine tasks and processes.

3. **Power Automate Integration**:

    - Purpose: Power Automate allows us to create automated workflows that can connect to Dataverse and other services.

    - Functionality:
        - Design complex workflows with conditional logic and branching.
        - Integrate with external systems and services.

4. **JavaScript Client-Side Code**:

    - Purpose: custom JavaScript code can be used to extend the client-side behavior of forms.

    - Functionality:
        - Perform custom validations and calculations.
        - Modify the appearance and behavior of forms.

5. **Plug-ins (Server-Side Code)**:

    - Purpose: Plug-ins are server-side code that can be executed in response to events in Dataverse.

    - Functionality:
        - Enforce complex business logic on the server.
        - Modify data before or after it is saved.

6. **Real-Time Workflow**:

    - Purpose: Real-time workflows are similar to traditional workflows but are executed in real-time.

    - Functionality:
        - Immediate response to events or conditions.
        - Execute business logic without delay.

7. **Field Validation Rules**:

    - Purpose: Field validation rules allow you to specify validation criteria for individual fields.

    - Functionality:
        - Ensure that data entered into a field meets specific conditions.
        - Display error messages for invalid data.

8. **Custom Business Logic (Code)**:

    - Purpose: For more complex or specific scenarios, you can write custom code using .NET and the Dataverse SDK (Software Development Kit).

    - Functionality:
        - Implement highly customized business logic.
        - Interact with external systems and services.

9. **Client-Side API**:

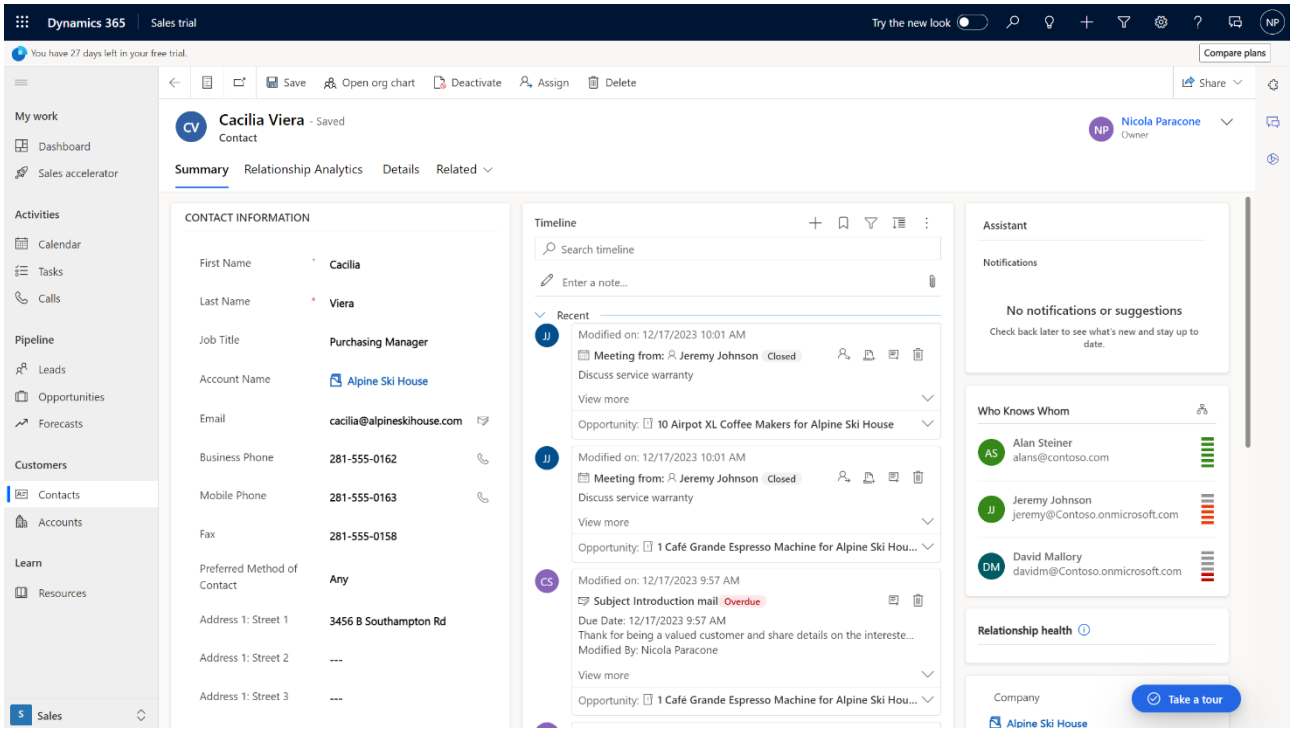
    - Purpose: Dataverse provides a client-side API that allows you to interact with data on forms using JavaScript.

    - Functionality:
        - Retrieve and manipulate data on the client side.
        - Perform custom validations and calculations.

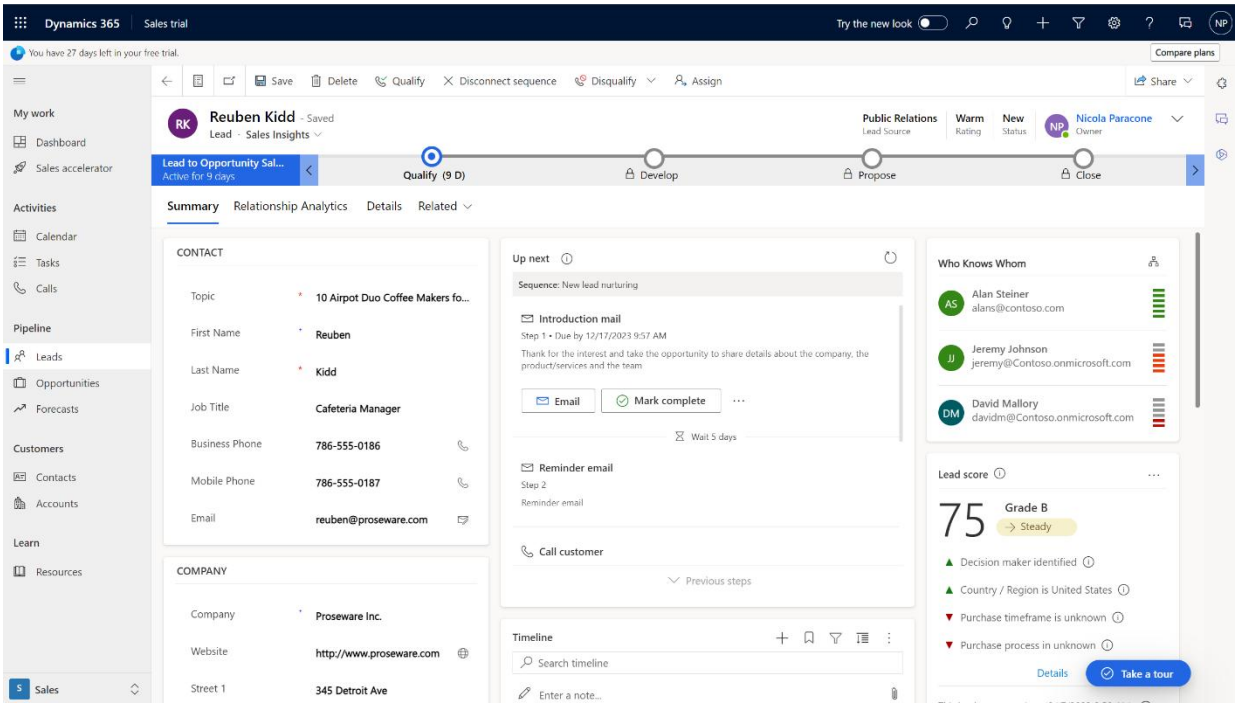
These tools and features collectively provide a robust set of options for implementing logic and validation in Dataverse, catering to various complexity levels and customization requirements. The choice of a tool depends on the specific business scenario and the level of customization and automation needed.

Some applicative examples follow.

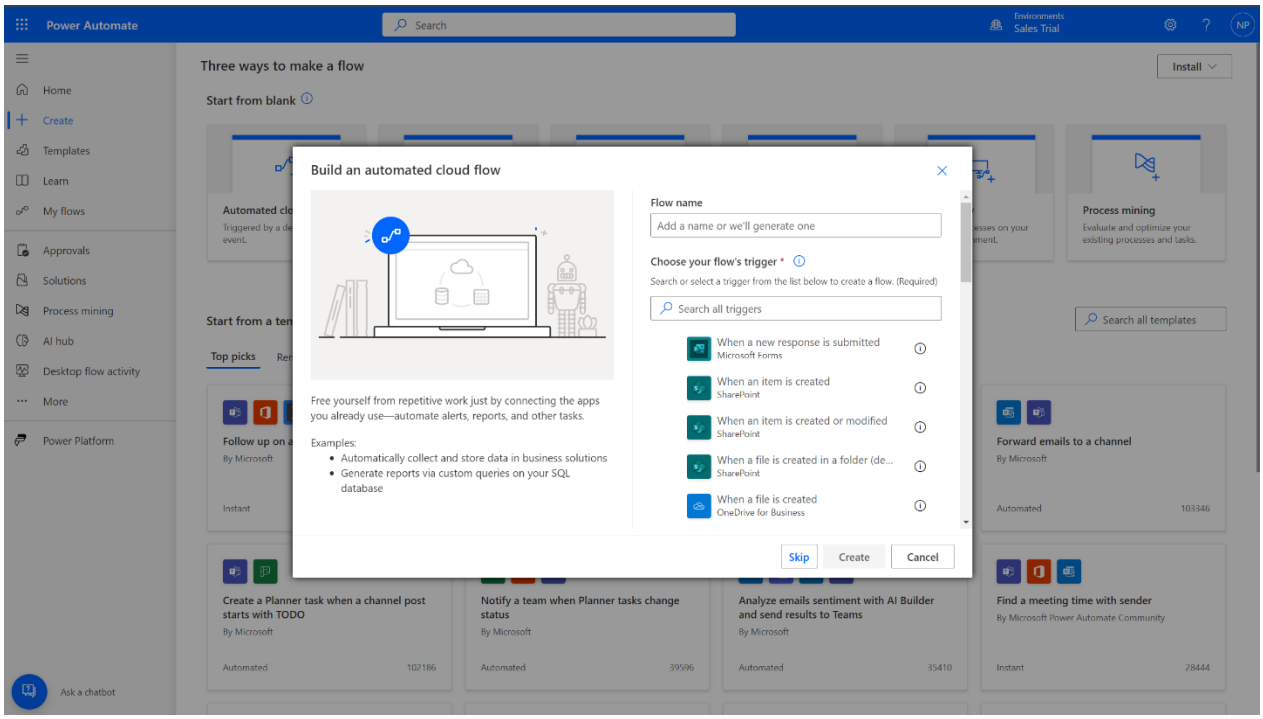
1. Assume that you want to create a new contact with the same email address as a contact already present. Before the contact is saved, we can ask the system to check if that email is already associated with a contact, and if that is the case stop the operation. Similarly, when a new contact is created, we potentially would like to create a related record, autocomplete some fields, or create a task for that user. For that we can create a workflow.



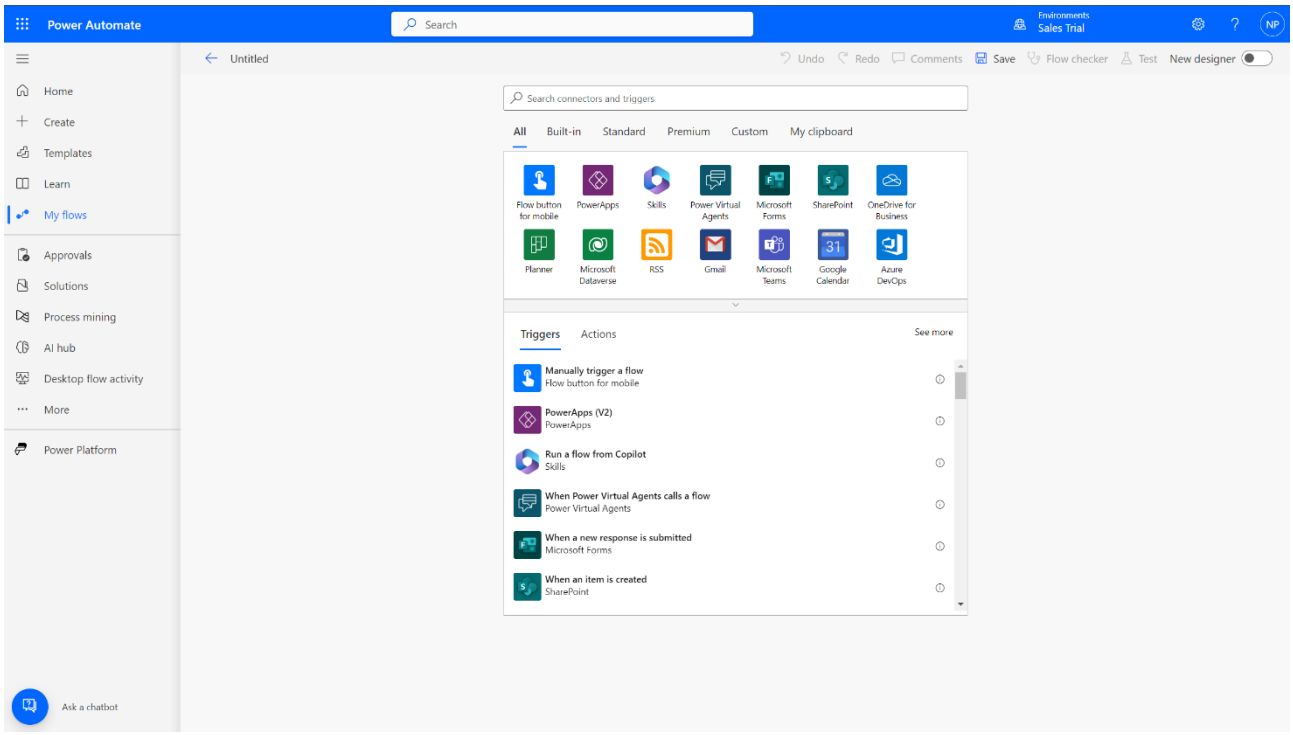
2. Assume that we have a *lead*, which is term referring to a record that represents a potential sales *opportunity*. Leads are often the initial stage in the sales process, representing individuals or organizations that have expressed interest in a product or service but have not yet been qualified as opportunities. The process typically involves converting leads into opportunities as they progress through the sales pipeline.



Employees in our company may want to qualify the lead and next proceed to the other steps: they are assigned a **business process flow**. By connecting to Power Automate (make.powerautomate.com) we can create our own processes, such as an Automated cloud flow (click on + Create) and connect to Dataverse.



Here we have to possibility to set triggers and actions to be performed (add a record, update a record, delete a record, get the Id of a contact etc.).



A **trigger** is an event that initiates the execution of a flow, an **action** refers to a specific operation or task that can be performed as part of an automated workflow. Actions are the building blocks of flows and represent individual steps that define what the flow does. Each action corresponds to a specific operation that interacts with a service, application, or data source.

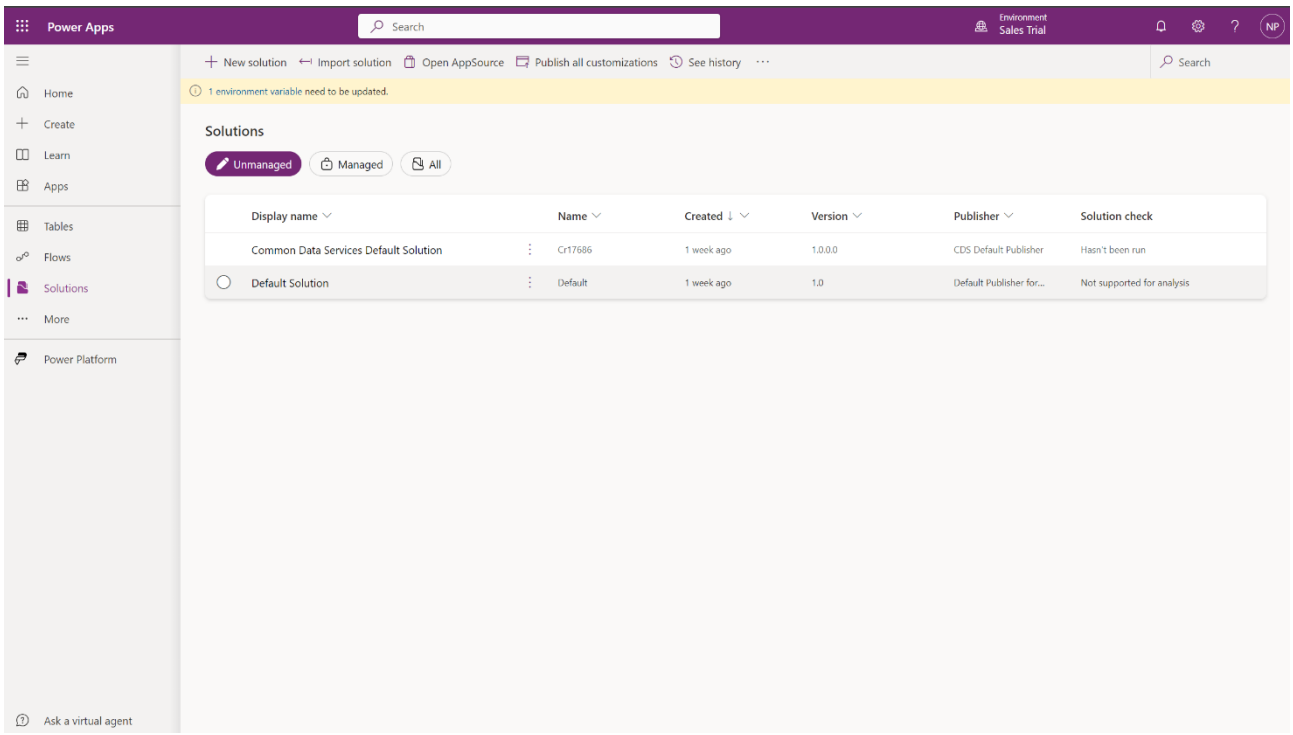
The advantage of doing these things within Dataverse is that the Dynamics 365 interface itself uses Dataverse as storage. Most of the times we will be working within Power Apps (make.powerapps.com), which also gives us the possibility of creating some processes and workflows, unfortunately with limited functionalities. Much more is accessible through Power Automate, but even in that case we will need to write custom codes for some particular scenarios.

Assuming that our data is to be shown to external users, we can use Power Pages: clients or customers can log in, see all the products and place orders. If a customer wants to know any particular order information, or we just want to assist the customer within our website, we will be using Virtual Agents. Based on the customer's questions, we will have some predefined answers (given the order number, for instance, we can create a flow to automatically provide the status of the order). Summarizing, using Dataverse we will be connecting to all five products of the Power Platform.

3. Create a solution

In Power Apps, a **solution** is a container that holds a set of components, such as apps, flows, entities, and more, which are used to organize and manage our applications and customizations. Solutions enable us to package, deploy, and manage Power Apps applications and assets across different environments.

Log in to Power Apps and select the right environment (Sales trial). Then go to Solutions (notice that there are both Managed and Unmanaged solutions) and click on + New Solution.



By default, a new solution is **unmanaged**. Components in an unmanaged solution remain directly editable within the environment where the solution is imported. Users can make changes to the components, add or remove items, and modify metadata directly in the environment. On the contrary, components in a **managed** solution are packaged and sealed, meaning their metadata is not directly editable after the solution is exported. Users cannot make changes to the components directly within the environment where the solution is installed. This helps protect the integrity of the solution and prevents unintended modifications by users.

In the real world we have three environments: **Dev**, **Test** and **Production**. In this scenario, we will usually export a solution and add it to another environment, usually Production, as Managed. Here are some key features:

1. **Development (Dev) Environment**:

   - Purpose: the Dev environment is primarily used for building and customizing applications. It is where developers create, modify, and test various customizations, configurations, and extensions.

   - Characteristics:
     - Unrestricted access to customization tools and features.
     - No restrictions on testing or experimenting with configurations.
     - Ideal for initial development and experimentation.

2. **Test Environment**:

   - Purpose: the Test environment is dedicated to testing and validating customizations, configurations, and solutions before they are deployed to the production environment. It is a controlled space for quality assurance.

   - Characteristics:
     - Mimics the production environment to ensure accurate testing.
     - Typically used for user acceptance testing (UAT) and performance testing.

- May contain a copy of production data for realistic testing scenarios.
- Access is restricted to authorized users to maintain the integrity of testing.

3. **Production Environment**:

   - Purpose: the Production environment is the live and operational instance of Dynamics 365 where end-users interact with the finalized and tested applications. It is the environment where business processes are executed, and live data is managed.

   - Characteristics:
     - Highly controlled and monitored to ensure stability and security.
     - Access is restricted to authorized users.
     - Customizations and configurations are typically limited to prevent unauthorized changes.
     - Integration with other systems and services for live business operations.

4. **Additional Considerations**:

   - **Data Migration**: when moving from one environment to another, especially from Test to Production, data migration processes may be necessary to ensure that the production environment has the most up-to-date data for ongoing operations.

   - **Solutions Deployment**: customizations and configurations developed in the Dev environment are typically packaged into solutions and then deployed to the Test and Production environments. This ensures consistency across different instances.

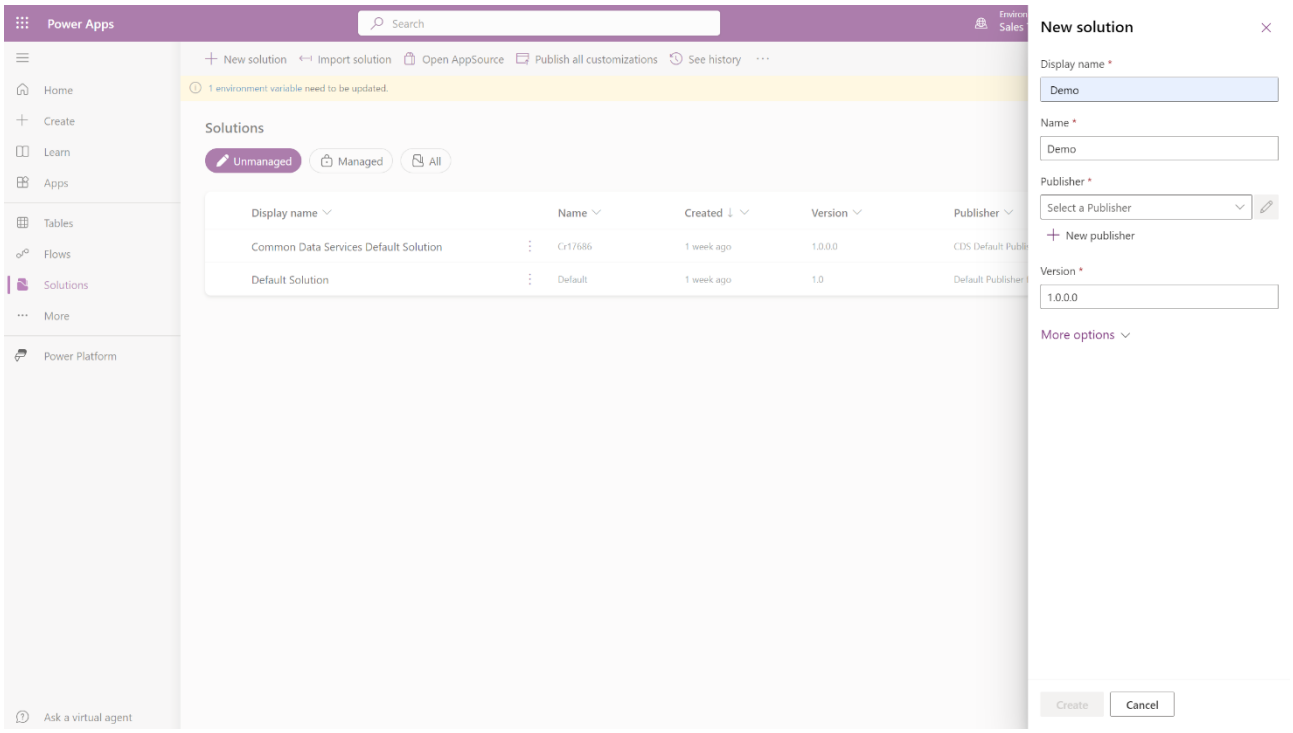
   - **Licensing**: each environment requires its own licensing, and it is important to adhere to licensing agreements to avoid compliance issues.

   - **Backup and Recovery**: regular backups are crucial, especially in the Production environment. This ensures that data can be restored in the event of unexpected issues, data loss, or system failures.

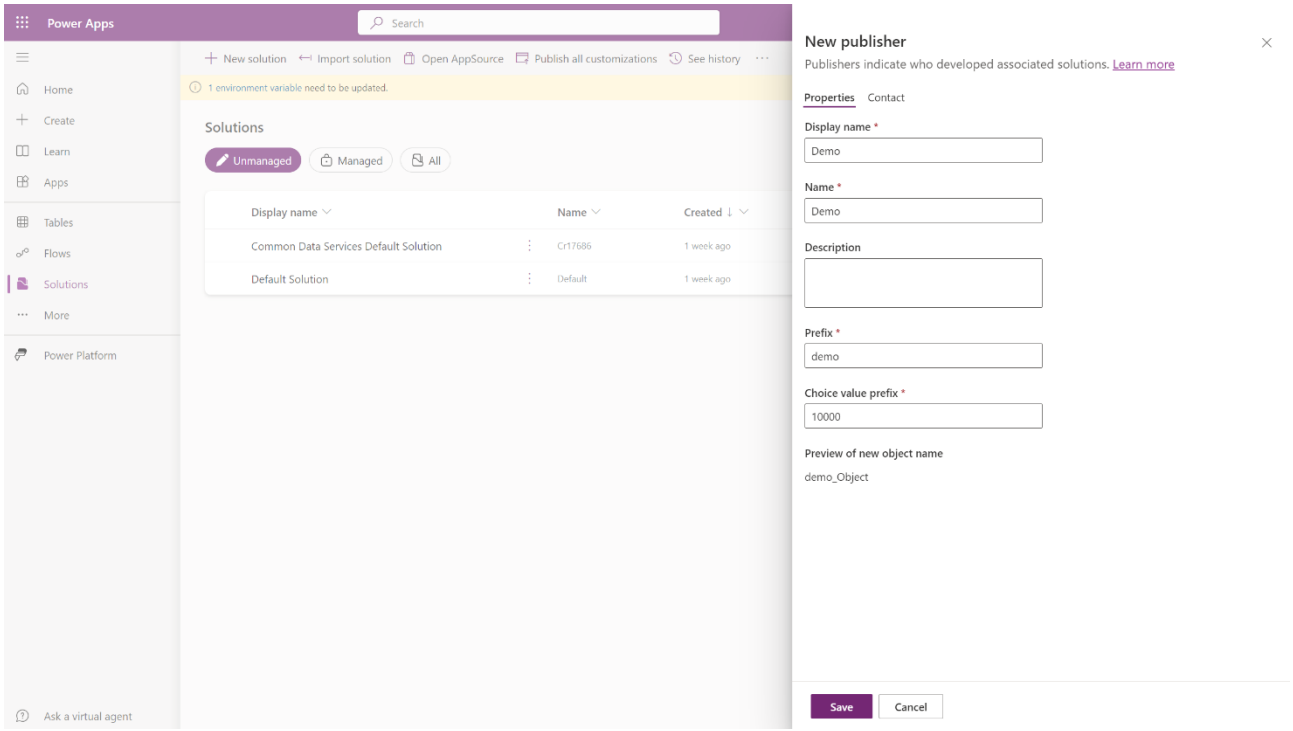
   - **Environment Types**: in addition to Dev, Test, and Production environments, Dynamics 365 supports additional environment types, such as *Sandbox* and *Trial*, each serving specific purposes in the application lifecycle.

By maintaining separate Dev, Test, and Production environments, organizations can implement a structured approach to development, testing, and deployment, reducing the risk of errors and ensuring a smooth transition from development to live production use.

We create a solution named Demo and create a custom Publisher (namely the organization which owns the solution).

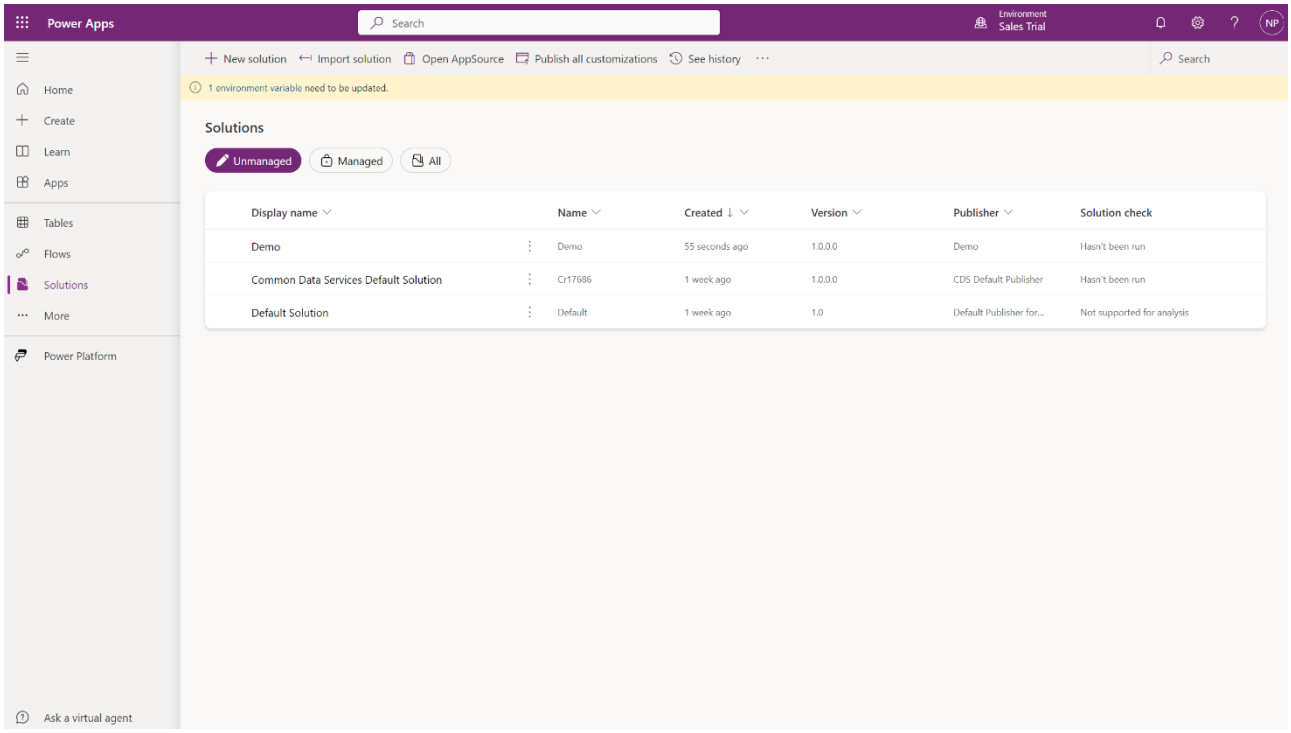


Publishers can be individuals, teams, or organizations responsible for managing and distributing customizations within the Power Platform. To create a custom solution publisher, we define a unique publisher name and a prefix that is used to create unique names for components, such as entities and fields, associated with the publisher.

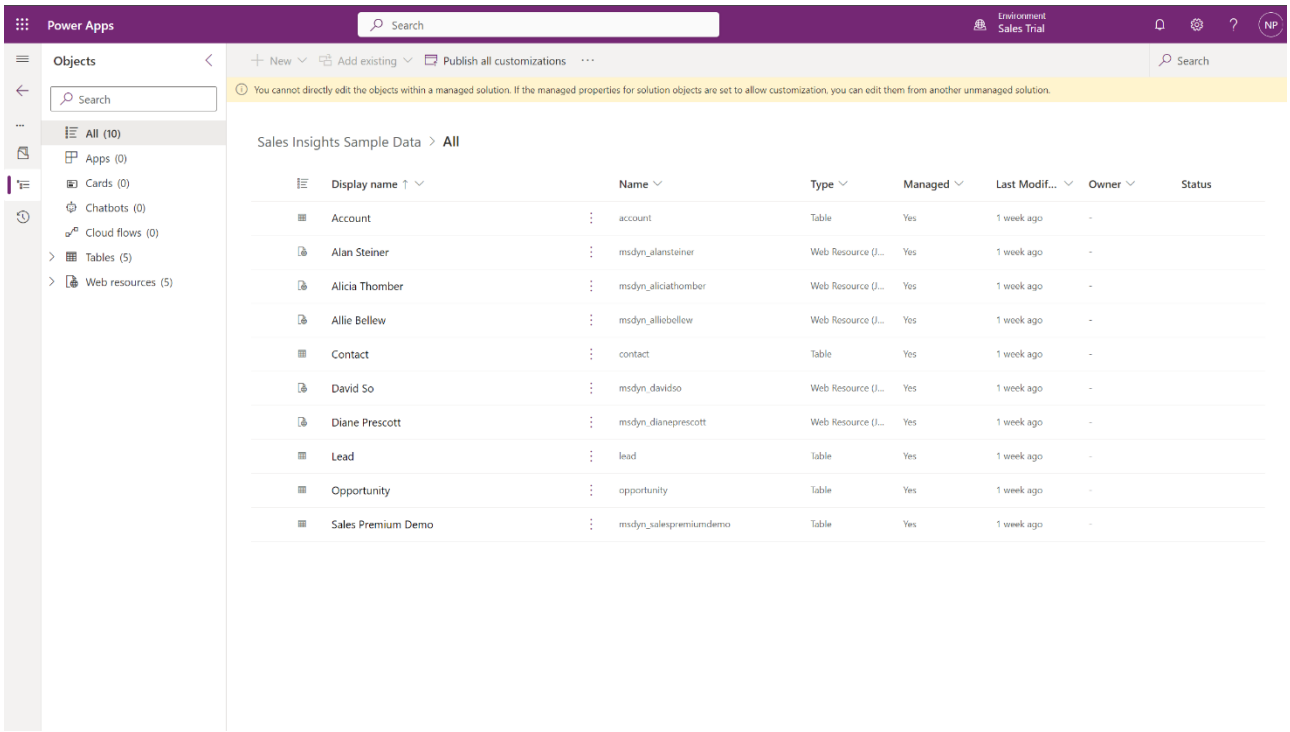


The prefix is a string of characters that is added to the names of custom entities, fields, and other components created by the solution publisher.

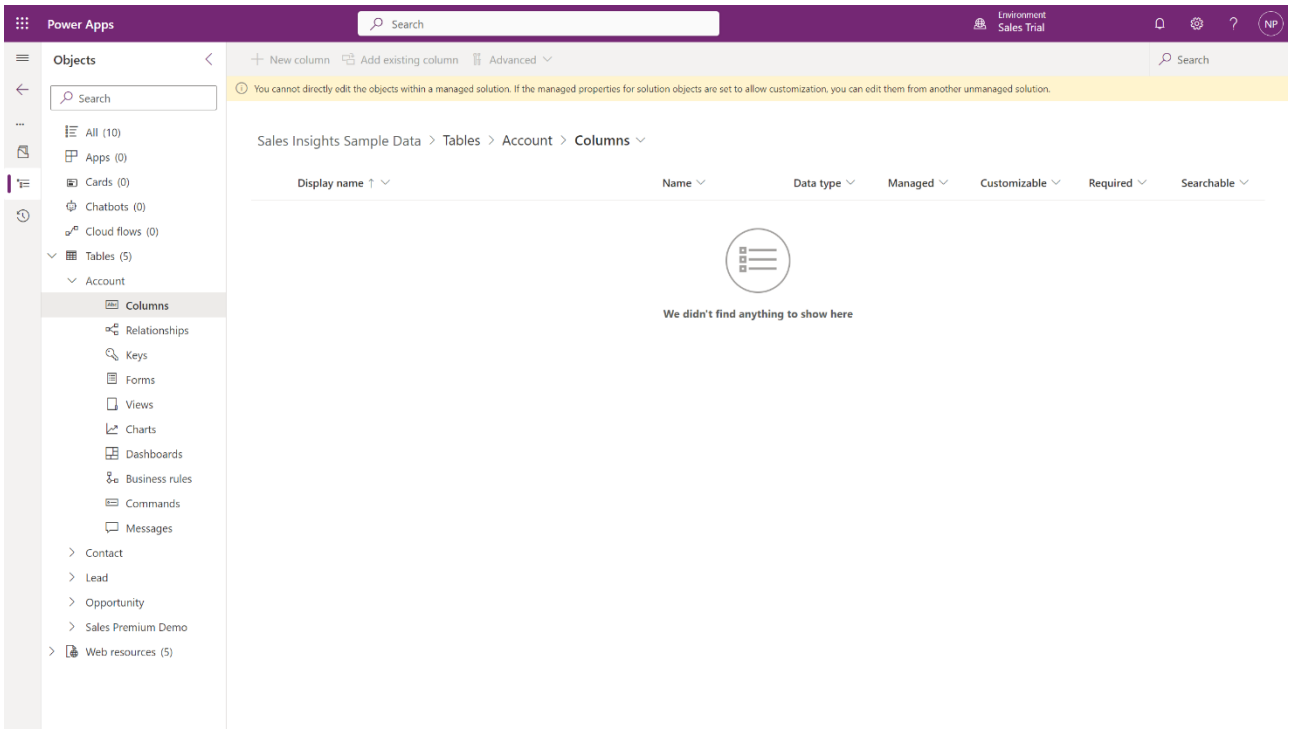
It helps ensure uniqueness and avoids naming conflicts with components from other publishers. The Choice value prefix values are used, for example, in choice fields.

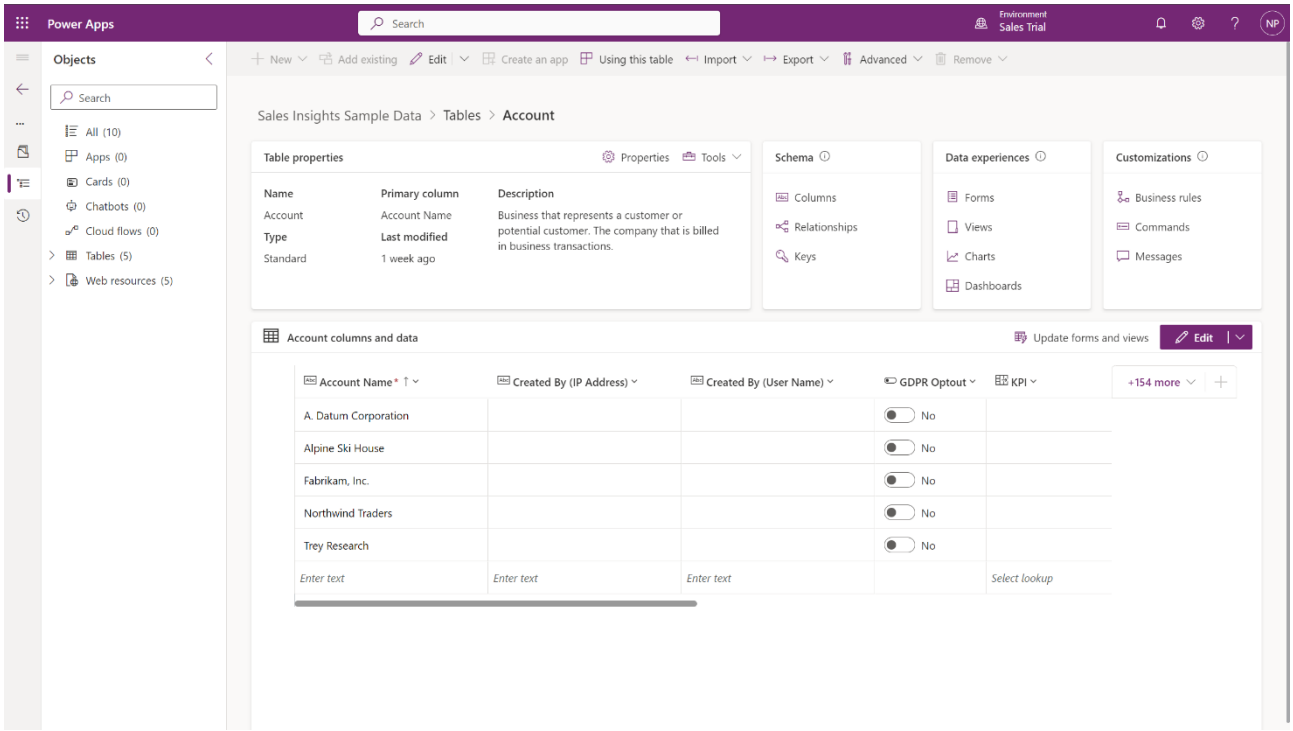


As we said, our solution will be unmanaged and therefore directly editable. On the contrary, if we open a managed solution in Power Apps a warning will be thrown.



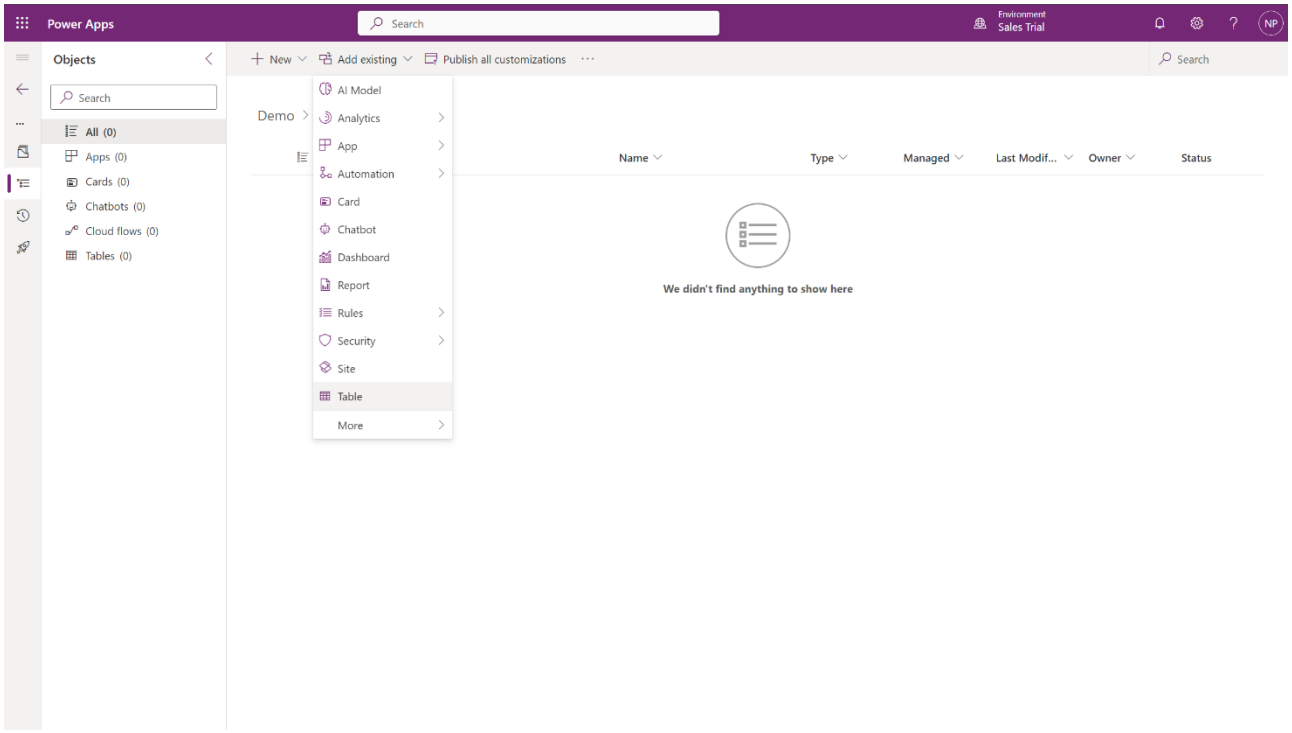
All the options for editing the solution will be disabled.


Power Apps
Search
Environment
Sales Trial
NP
Objects
Search
All (10)
Apps (0)
Cards (0)
Chatbots (0)
Cloud flows (0)
Tables (5)
Account
Columns
Relationships
Keys
Forms
Views
Charts
Dashboards
Business rules
Commands
Messages
Contact
Lead
Opportunity
Sales Premium Demo
Web resources (5)
New column
Add existing column
Advanced
Search
You cannot directly edit the objects within a managed solution. If the managed properties for solution objects are set to allow customization, you can edit them from another unmanaged solution.
Sales Insights Sample Data > Tables > Account > Columns
Display name
Name
Data type
Managed
Customizable
Required
Searchable
We didn't find anything to show here

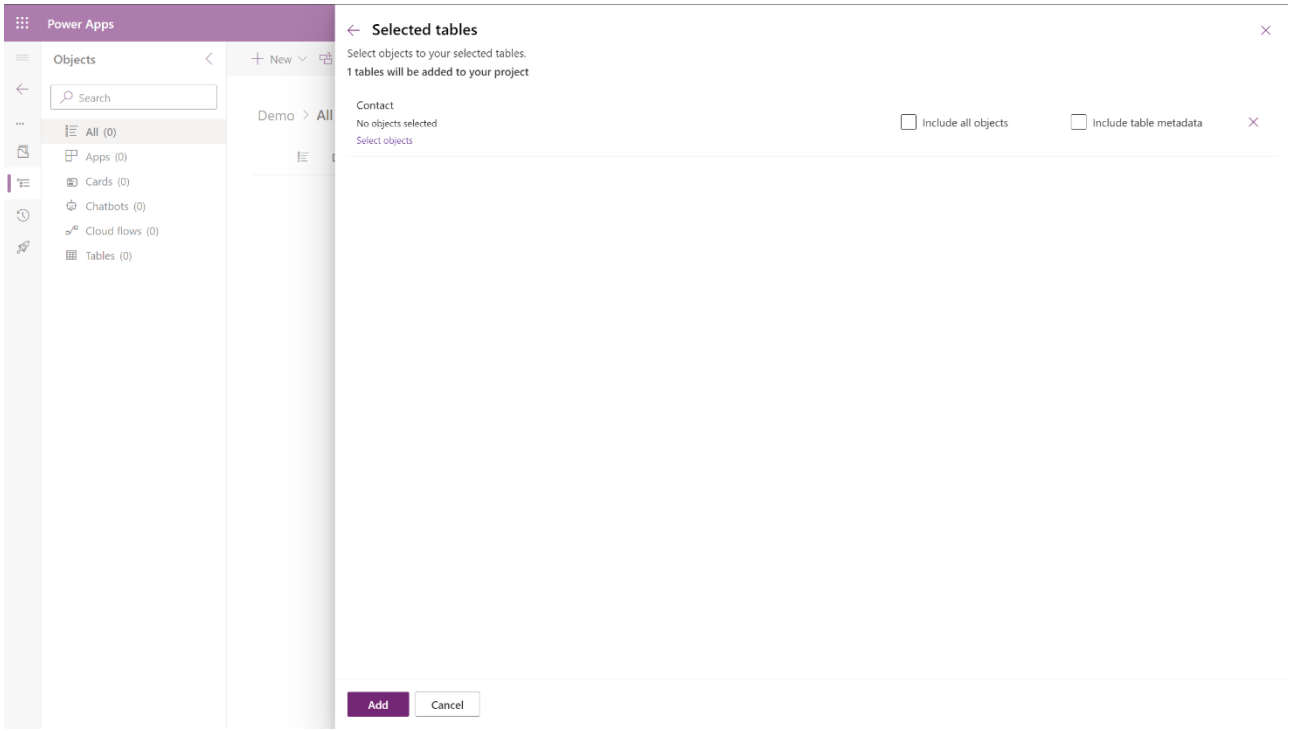

Power Apps
Search
Environment
Sales Trial
NP
Objects
Search
All (10)
Apps (0)
Cards (0)
Chatbots (0)
Cloud flows (0)
Tables (5)
Web resources (5)
New
Add existing
Edit
Create an app
Using this table
Import
Export
Advanced
Remove
Sales Insights Sample Data > Tables > Account
Table properties
Properties
Tools
Name
Account
Primary column
Account Name
Description
Business that represents a customer or potential customer. The company that is billed in business transactions.
Type
Standard
Last modified
1 week ago
Schema
Columns
Relationships
Keys
Data experiences
Forms
Views
Charts
Dashboards
Customizations
Business rules
Commands
Messages
Account columns and data
Update forms and views
Edit
Account Name *
Created By (IP Address)
Created By (User Name)
GDPR Optout
KPI
+154 more
A. Datum Corporation
Alpine Ski House
Fabrikam, Inc.
Northwind Traders
Trey Research
No
Enter text
Select lookup

5. Add a table to the solution

Let us add to our Demo solution an existing table already provided by Microsoft, for example the Contact table.



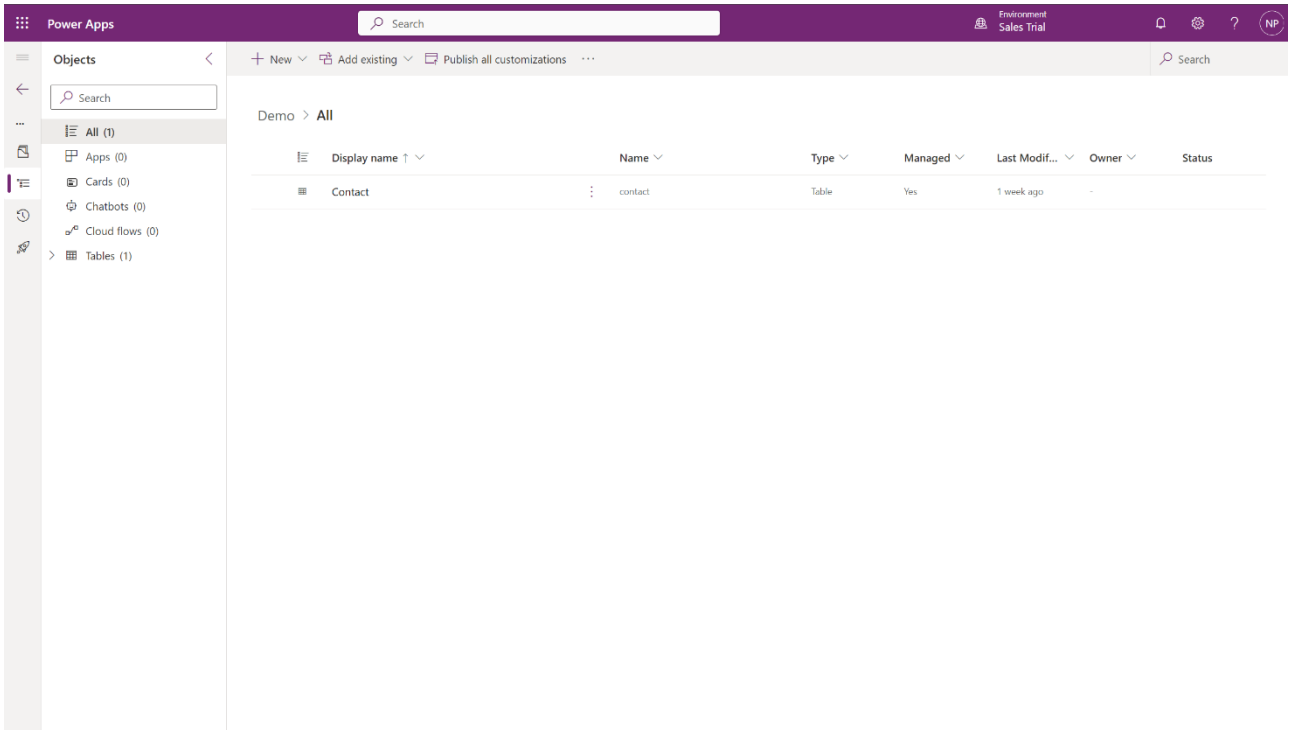
There we have to choose among two things: Include all objects and Include table metadata.



Include table metadata means including the properties of the table, like schema name, primary column and all functionalities (like quick creating of orders without leaving the contact view). Include all objects means all columns, views, relationships and business rules. This means that also the table metadata are imported.

Alternatively, we can select the objects we want. This is used if we want to move our data to the production environment. Instead of moving all the fields, we decide which fields are to be moved.

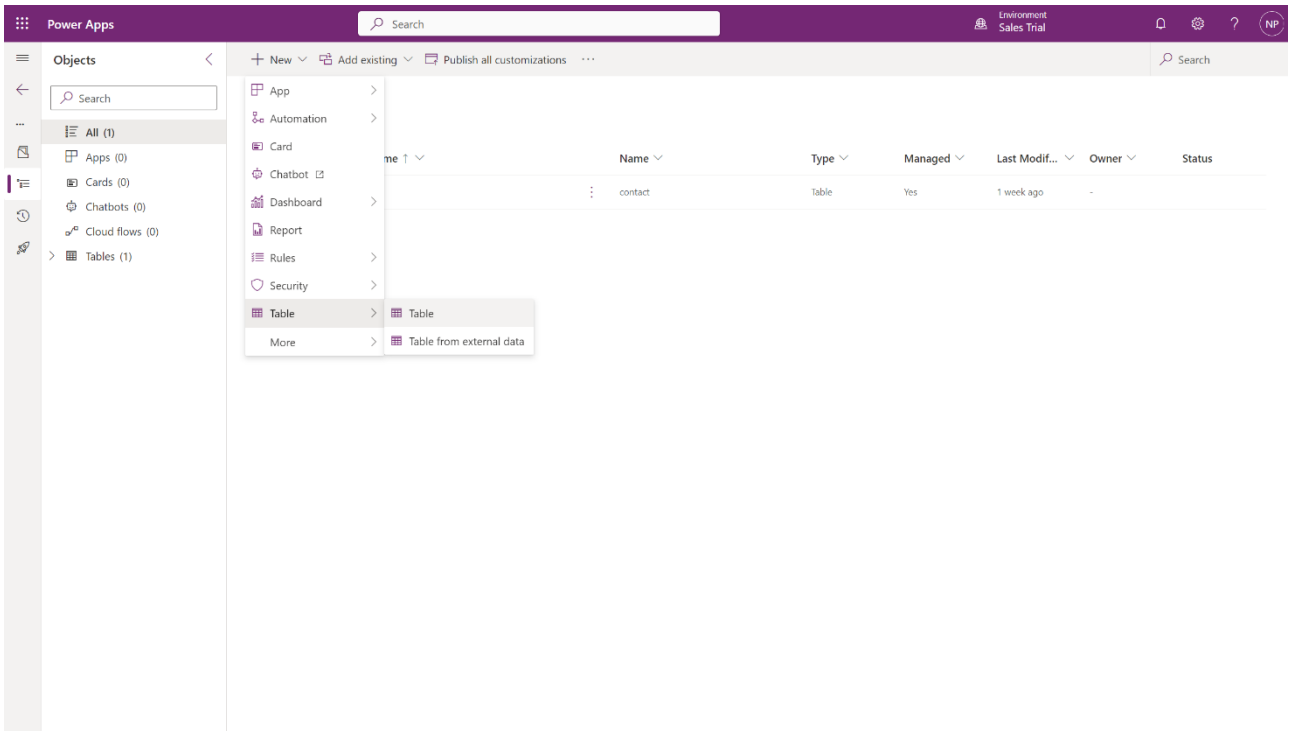
In our case we choose to import the Contact table with all its table metadata and click Add.



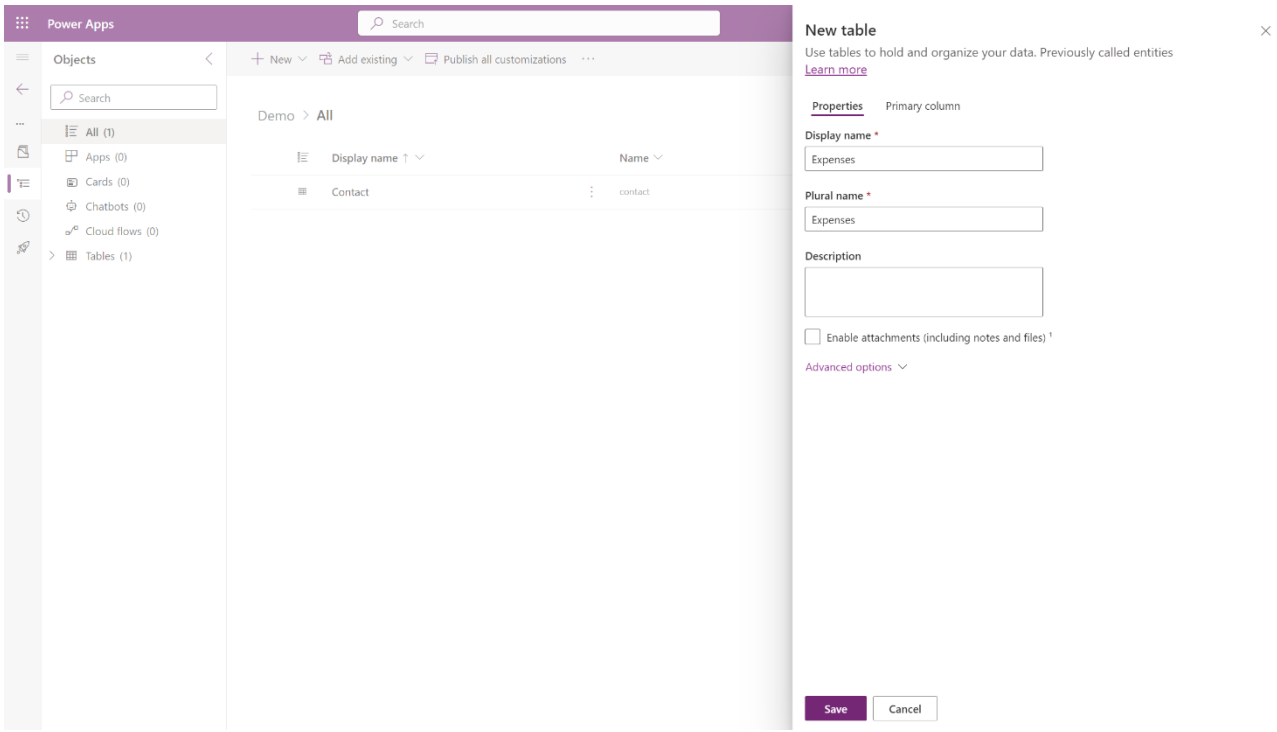
Good practice: all tables, workflows, etc., should be contained in one main solution.

6. Create a new table inside the solution

Now we create a new table. Click on + New and choose Table and Table again.

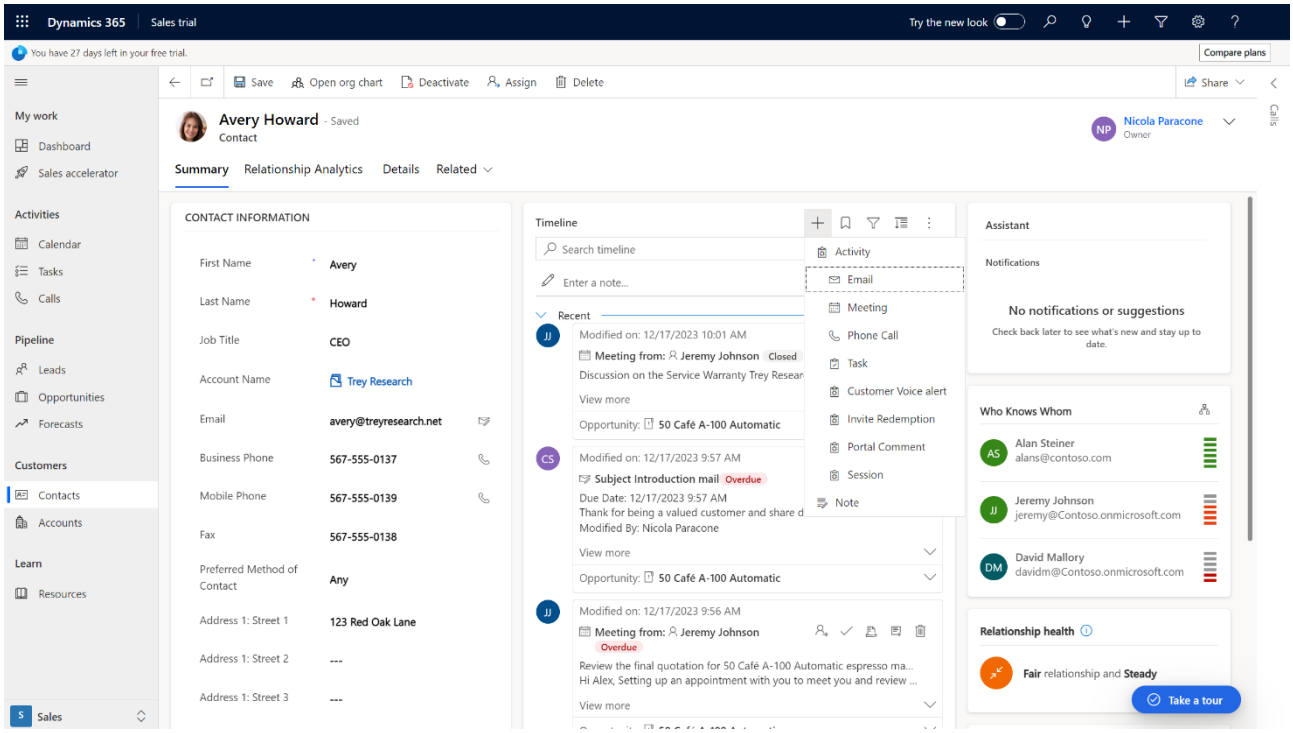


We name the new table Expenses.

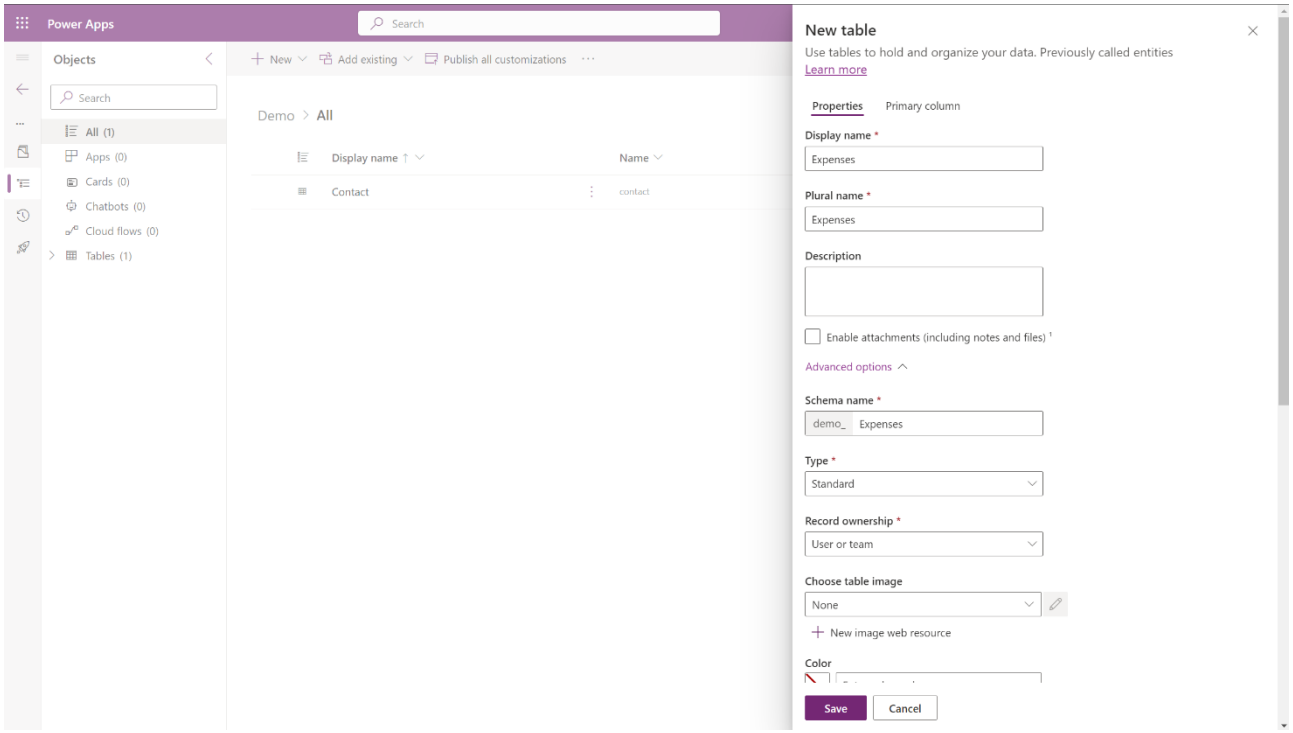


The purpose is the following: a contact may need to submit its expenses and have a place where to retrieve them all.

We have the possibility to Enable attachments, which activates the Timeline functionality. With this we can create new Activities. If we want the user to upload documents and files, then we enable this feature.



In the Advanced options section we have the Schema Name, where demo_ indicates the publisher used in creating the solution. demo_Expenses is the **logical name** of table. We also have Type, which indicates what type of entity we are trying to create. In our case we set it to Standard.



Here is what each entity type generally represents:

1. **Standard Entity**:

    - A standard entity is a type of entity that represents a standard business data structure. These entities are designed to handle typical business data such as accounts, contacts, opportunities, and other standard business entities.

    - Standard entities come pre-defined in the system and often serve as building blocks for common business scenarios.

2. **Activity Entity**:

    - An activity entity represents an activity or interaction, such as emails, phone calls, appointments, tasks, etc. These entities are used to track and manage communication and tasks within the system.

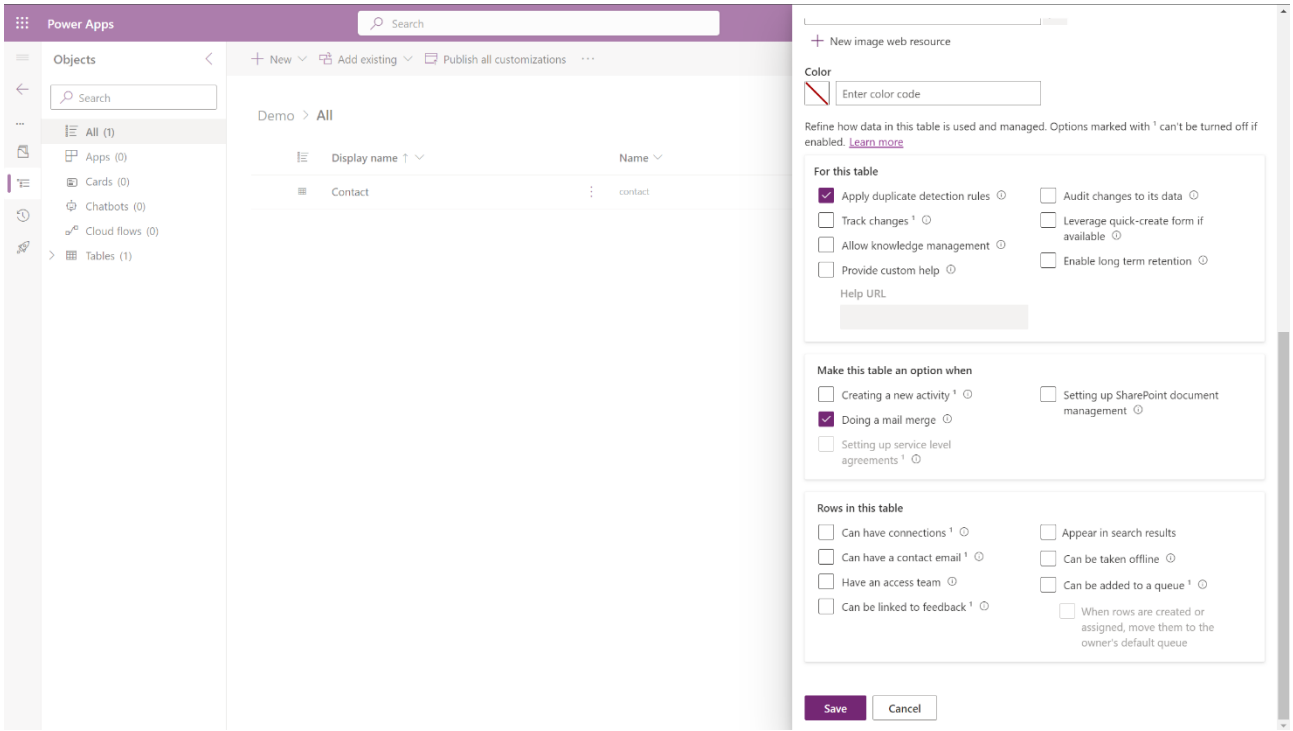
    - Examples include Email, Phone Call, Appointment, etc.

3. **Custom Entity**:

    - A custom entity is one that is created by users or administrators to represent specific data structures that are unique to a particular organization or business scenario.

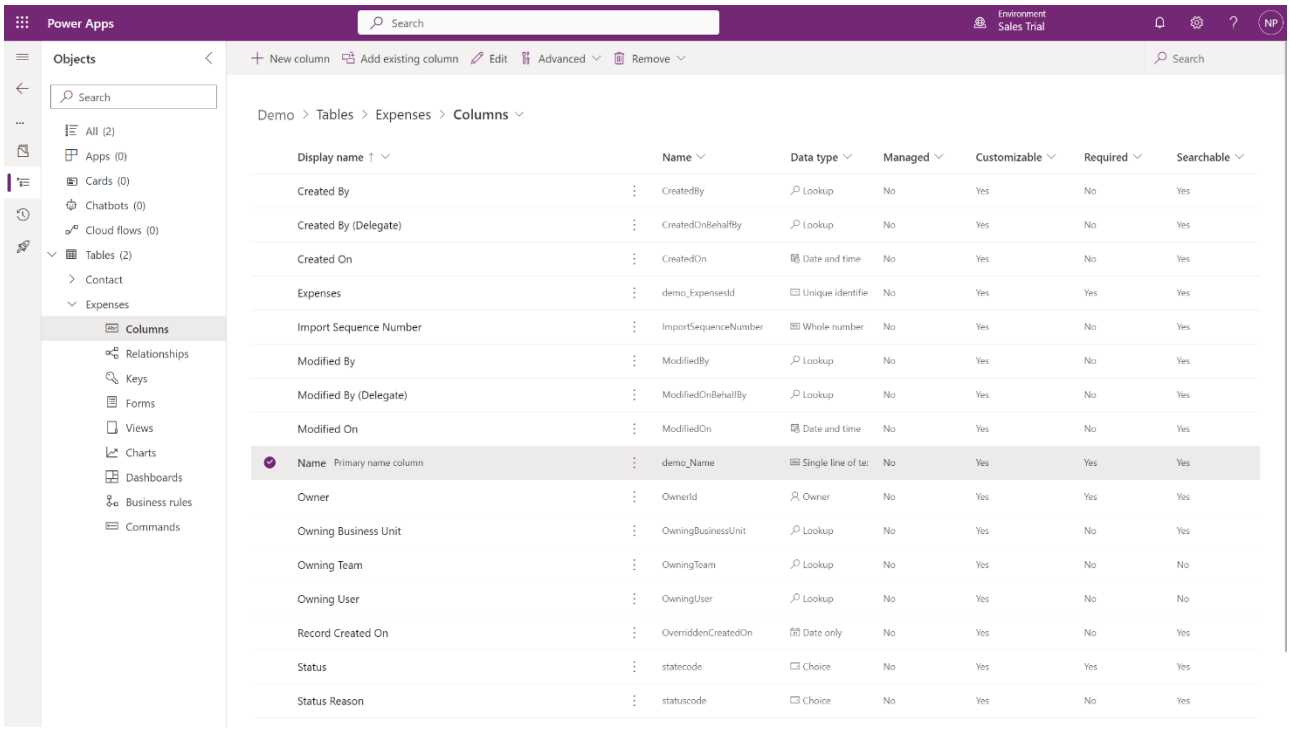
    - Custom entities are defined to meet specific requirements that are not covered by standard or activity entities.

Notice that we can mark other options, such as Track changes, which is self-explanatory.



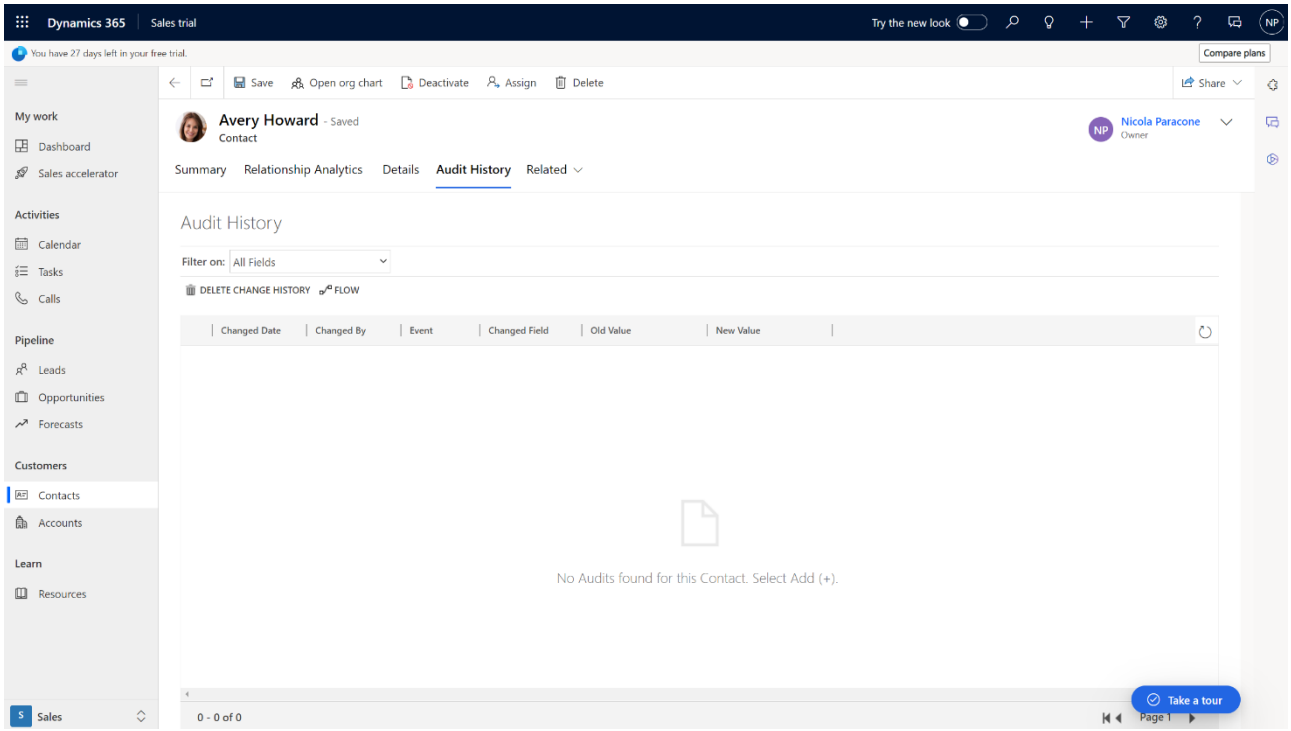
Then we save the table. So far, we have created a Demo solution and within it we have embedded the pre-existing Contact table and the newly created Expenses table.

When we create a new custom table, Microsoft will give us some default fields or attributes.



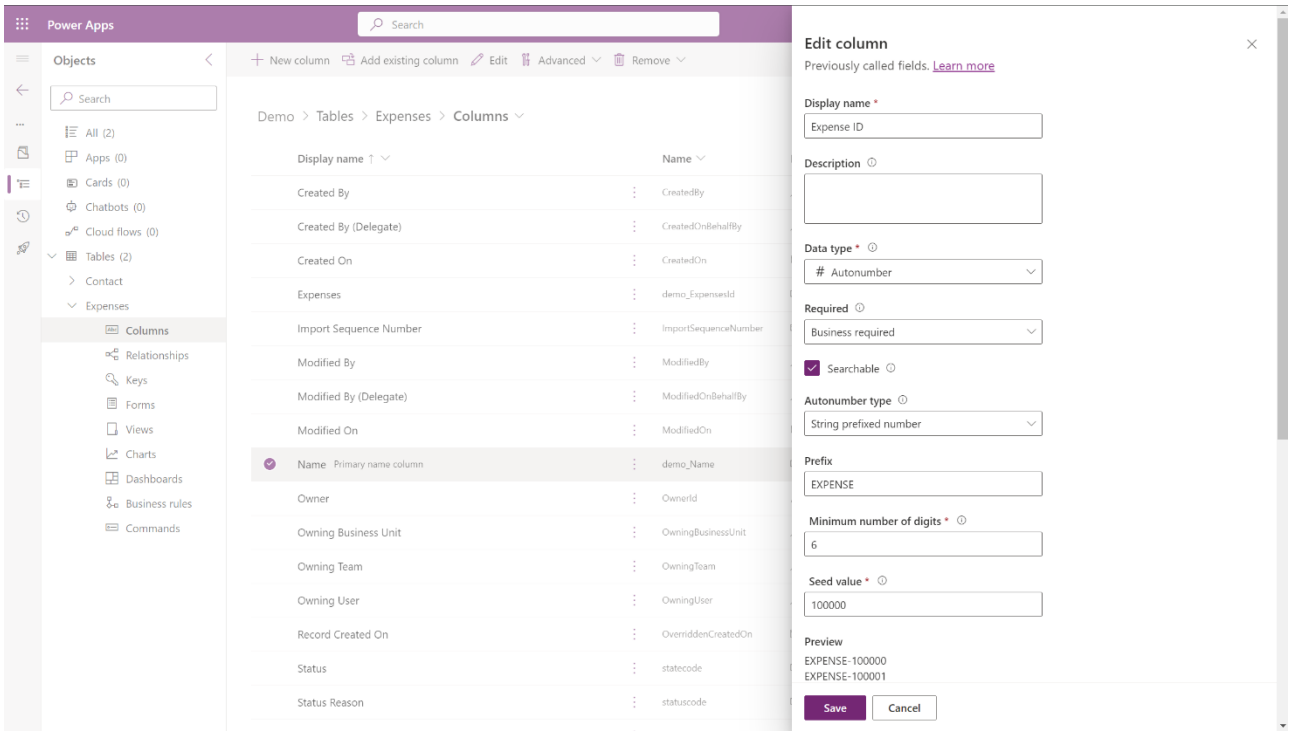
Created By, for instance, indicates which user has created a particular record (useful to track who created an order or invoice).

Assuming that auditing is enabled by our organization, we can go to a contact, click on Related and Audit History.



Here it captures all the modifications which have been done (it also shows old and new values of the modified fields).

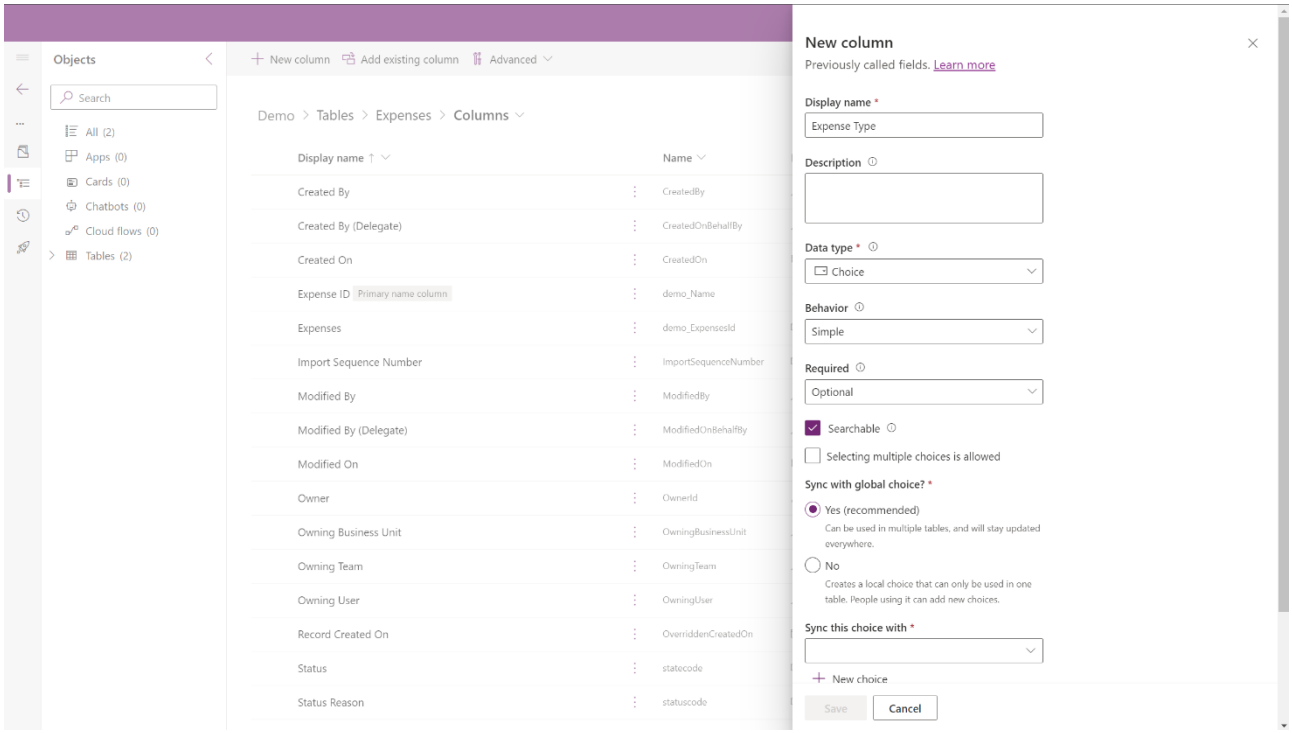
Primary name column has the functionality of adding a hyperlink which automatically opens the record. By default, Name is the primary name column. We rename it into Expense ID.



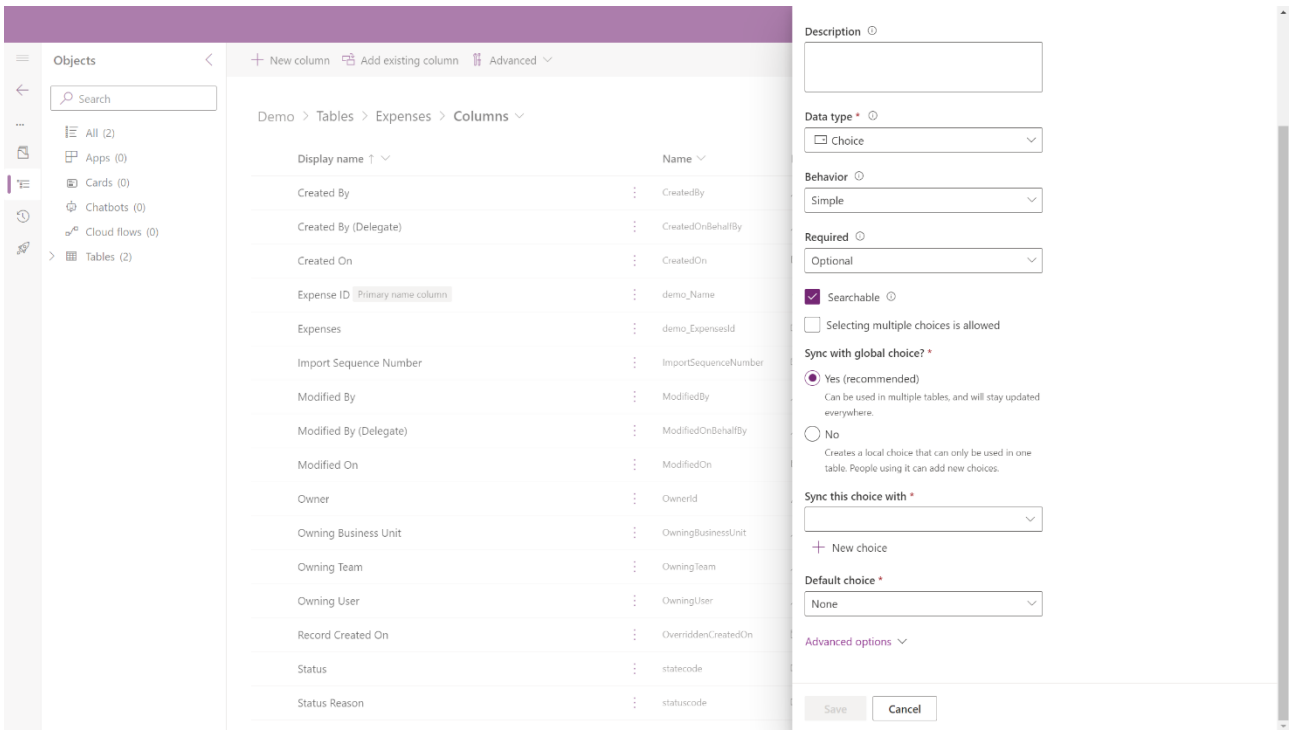
As data type we choose Autonumber. Based on the previous expense record, the system will automatically generate a number to be associated with the new record. Then we can also edit the autonumber type. In our case we choose String prefixed number, with Prefix = EXPENSE, Minimum number of digits = 6 and Seed value = 100000. This Id will be used to track all expense records.

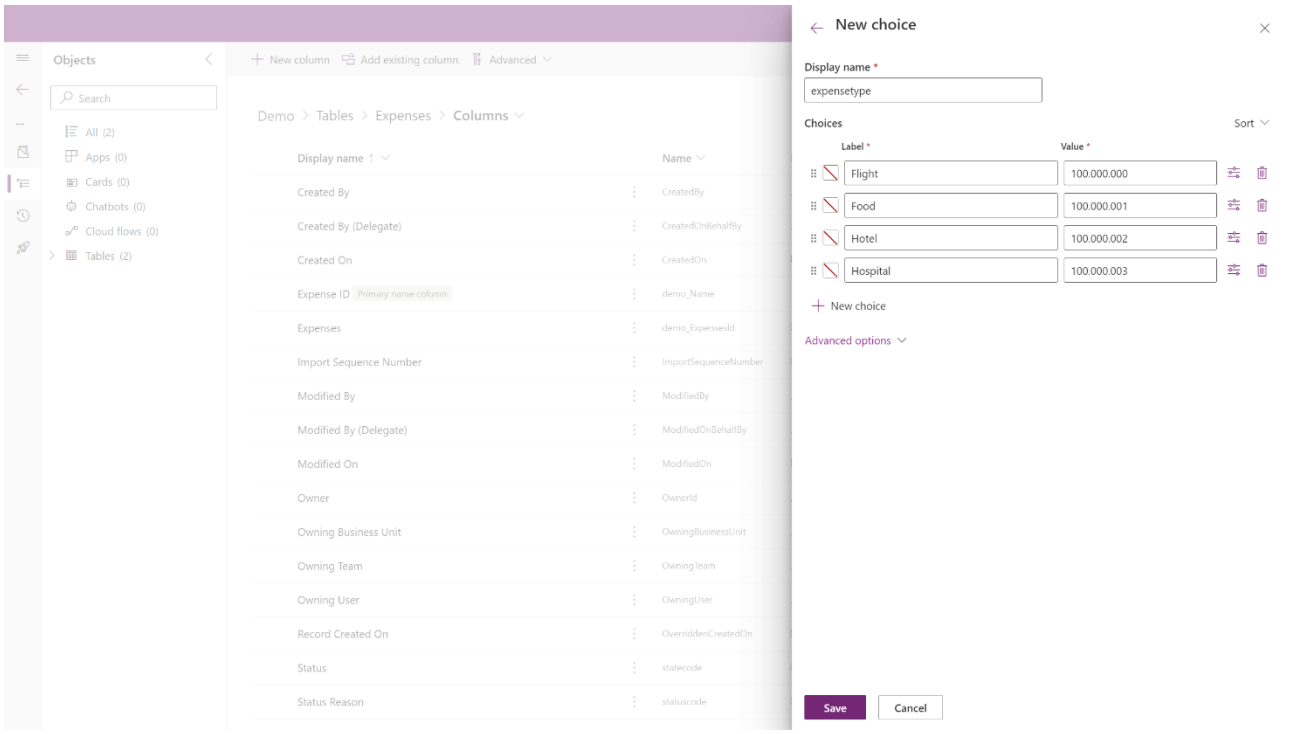
The GUID (global unique identifier) is used instead to uniquely identify a record. It is a thirty-two digits number present in the URL associated with that particular record. This is true for all kinds of tables.

Now we add a new column called Expense Type of type choice.



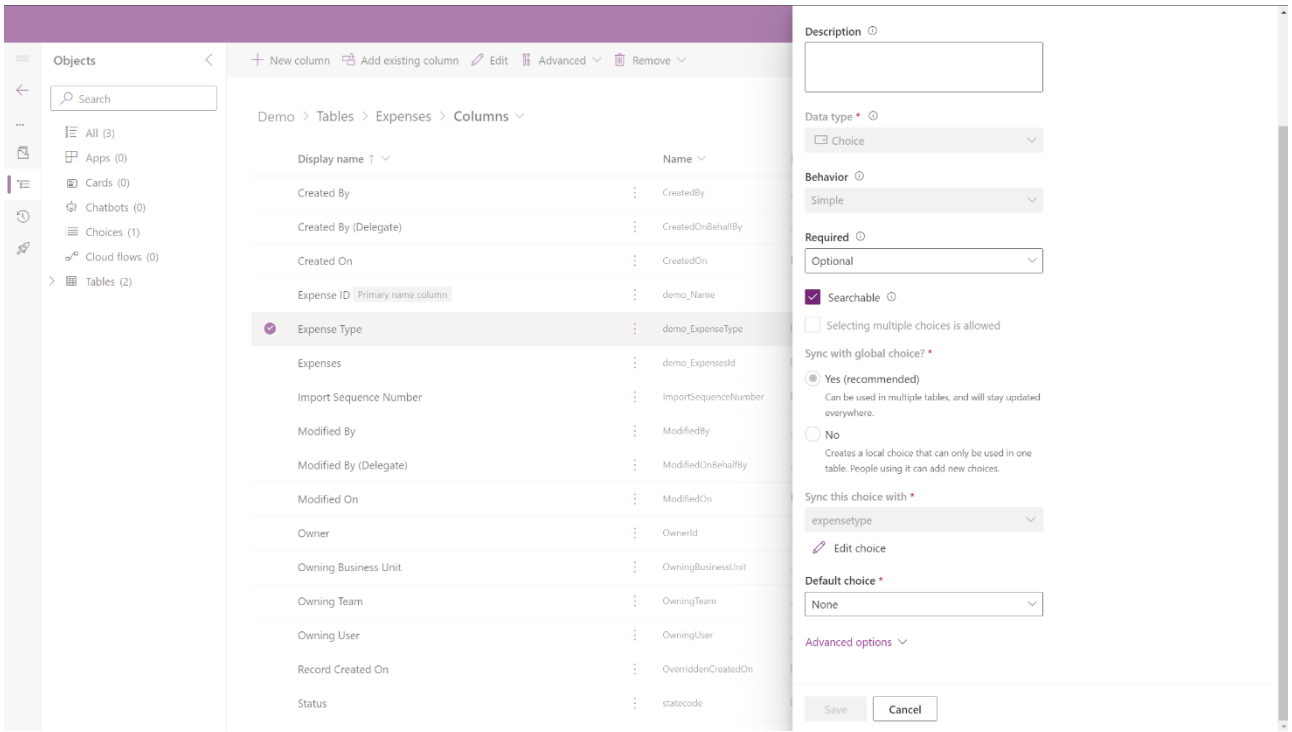
Since we do not have any predefined choice in this scenario, we go on and create our own custom choices.





Click on Save, and then select expensetype into the bar associated with Sync this choice with.

Notice that we also have the possibility to set a default choice.



A **lookup field** is a type of field that allows to establish a relationship between two entities. A lookup field enables us to reference a record from another entity within the current entity, creating a connection between the two records. This relationship is established by linking the lookup field to a specific entity type.

Here are key characteristics and aspects of lookup fields in Dynamics 365:

1. **Reference to Another Entity**:
    - A lookup field allows us to reference a record in another entity. For example, we might have a Customer entity and use a lookup field to associate a specific customer with a record in another entity, such as an Opportunity or a Case.
2. **User-Friendly Representation**:
    - Lookup fields provide a user-friendly representation of the linked record, typically displaying a clickable link or dropdown list that allows users to select the related record.
3. **Configurable Display Name**:
    - The display name of the lookup field can be configured to show a meaningful label that reflects the relationship. For example, a lookup field named Primary Contact might be used to associate a contact with an account.
4. **Establishing Relationships**:
    - Lookup fields play a crucial role in establishing relationships between entities. For example, a Parent Account lookup field in the Account entity might link an account to its parent account.
5. **Cross-Entity Relationships**:
    - Lookup fields enable us to create relationships across different entities. This is valuable for connecting records in different business contexts.
6. **Navigating to Related Records**:
    - Users can often click on the lookup field to navigate directly to the related record. This provides a convenient way to access and view details of the linked record.
7. **Configuring Behavior**:
    - Lookup fields can be configured to define the behavior when the linked record is deleted or deactivated. For example, you might choose to delete records that are no longer associated with a lookup field.
8. **Used in Views and Reports**:
    - Lookup fields can be utilized in views and reports to display information from the linked records, providing a comprehensive view of data.
9. **Support for Hierarchical Relationships**:
    - In some scenarios, lookup fields may be used to create hierarchical relationships, such as linking records to their parent or child records.

7. Create a model-driven app

At this point, we create a model-driven app and add to it the two tables. A **model-driven app** in Power Apps is a type of business application that is built on top of the Power Platform. Model-driven apps are designed to streamline business processes and provide a user interface that is automatically generated based on the underlying data model. These apps are often used for more complex business scenarios where data structures are well-defined, and the focus is on enforcing business rules and processes.

Here are key features and aspects of model-driven apps:

1. **Data-Driven Design:**
   - Model-driven apps are based on a data model that defines the structure and relationships of data entities (tables).
   - The user interface is generated automatically based on this data model.
2. **Components:**
   - Model-driven apps consist of components such as forms, views, charts, and dashboards.
   - Forms are used to display and edit records, views define how data is displayed, and charts and dashboards provide visual representations of data.
3. **Business Rules and Processes:**
   - Model-driven apps allow us to define business rules and processes using the Power Platform's business process flows.
   - Business rules can enforce data validation, automate tasks, and guide users through specific processes.
4. **Security:**
   - Security roles and permissions are an integral part of model-driven apps.
   - Users are granted specific roles that determine their access to entities, fields, and other components within the app.
5. **Integration:**
   - Model-driven apps can be integrated with other Power Platform services, such as Power Automate for workflow automation and Power BI for analytics.
   - Integration with external data sources and services is also possible.
6. **Responsive Design:**
   - Model-driven apps are designed to be responsive, providing a consistent user experience across different devices and screen sizes.

7. **Common Data Service (CDS):**

- Model-driven apps often leverage the Common Data Service (CDS), a data platform that allows you to securely store and manage data used by business applications.

- CDS provides a common data model, making it easier to integrate and share data between different applications.

8. **Customization:**

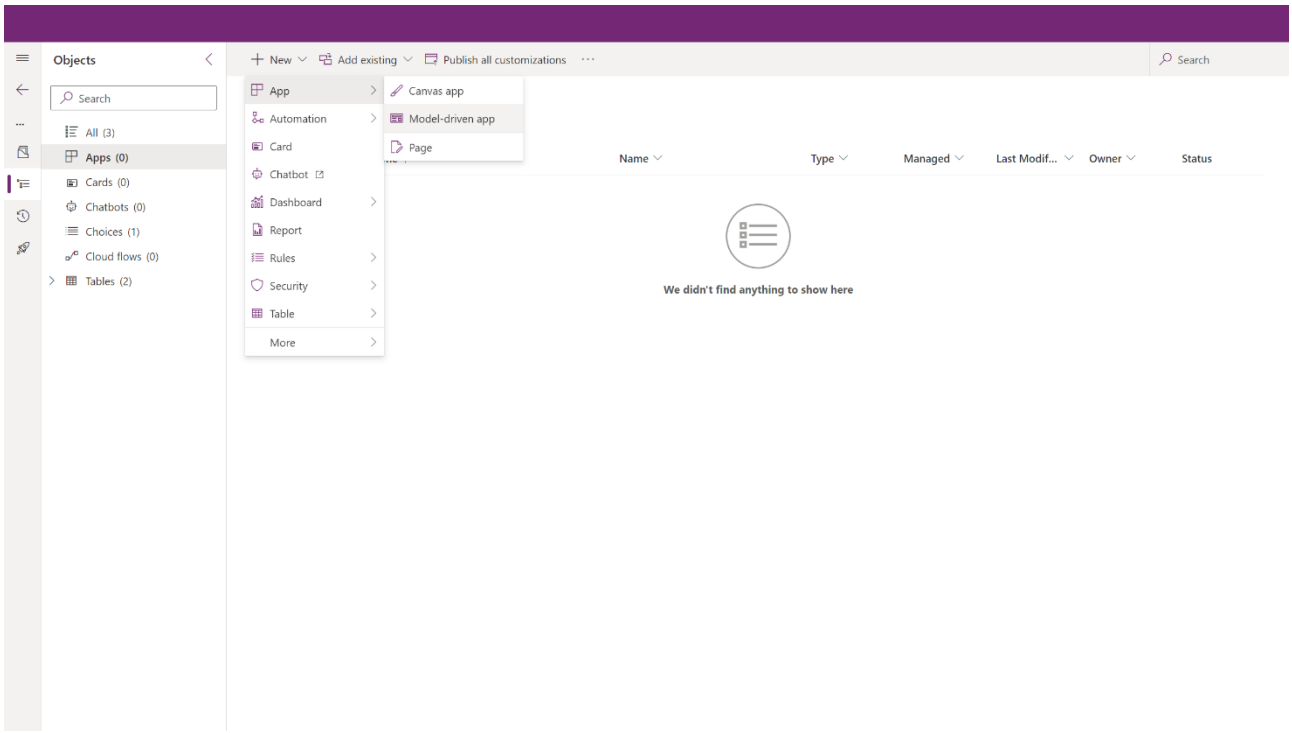
- Model-driven apps are customizable, allowing you to tailor the user interface, forms, and views to meet specific business requirements.

- Customizations can be done using the Power Apps Maker portal.

Model-driven apps are suitable for scenarios where there is a need for a structured and standardized approach to business processes, data management, and user interfaces. They are particularly well-suited for scenarios such as customer relationship management (CRM), project management, and other data-centric applications.

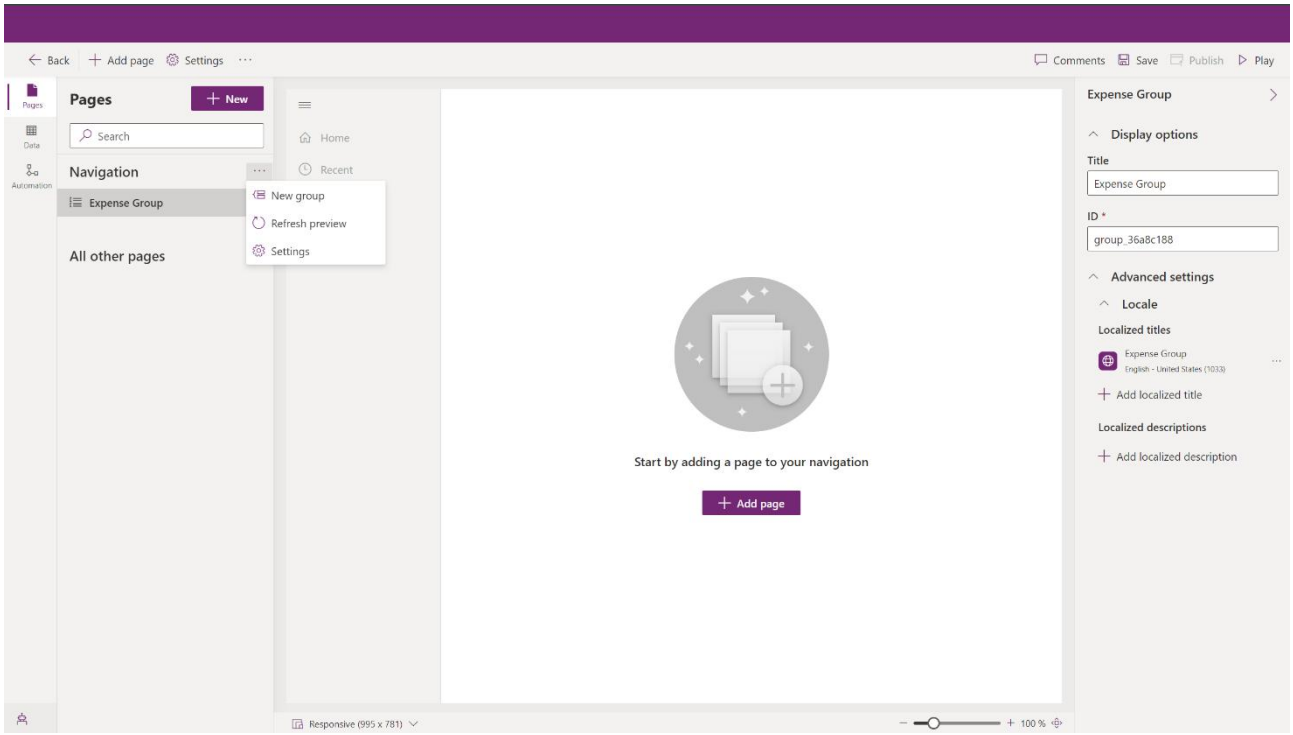
Sales trial is itself an app.

In Power Apps we go to the Demo solution and click on Apps, App and then Model-driven app.

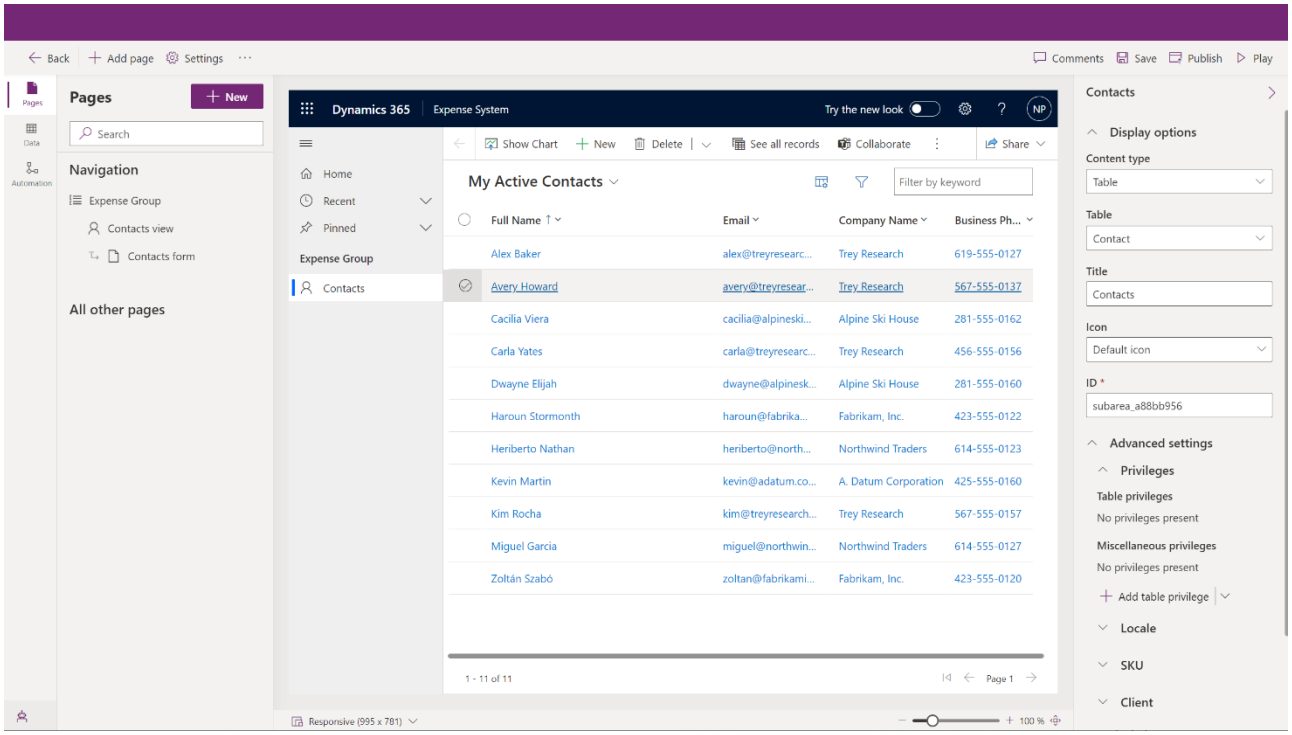


Here we must choose a name, for example Expense System. This app will give a user the ability to see all the existing expense records, or add new records for any contact.

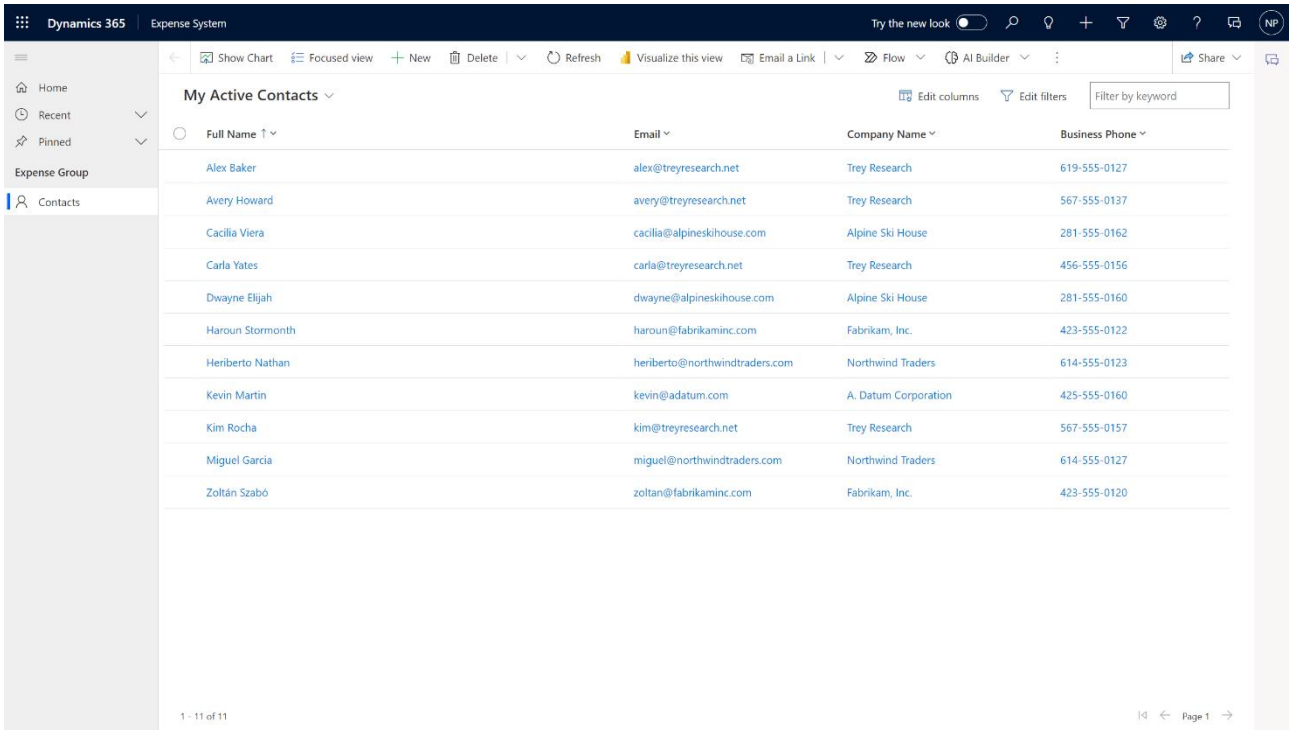
When we first create the app, we have to add a subarea by clicking on Navigation and selecting New group. We name it Expense Group.



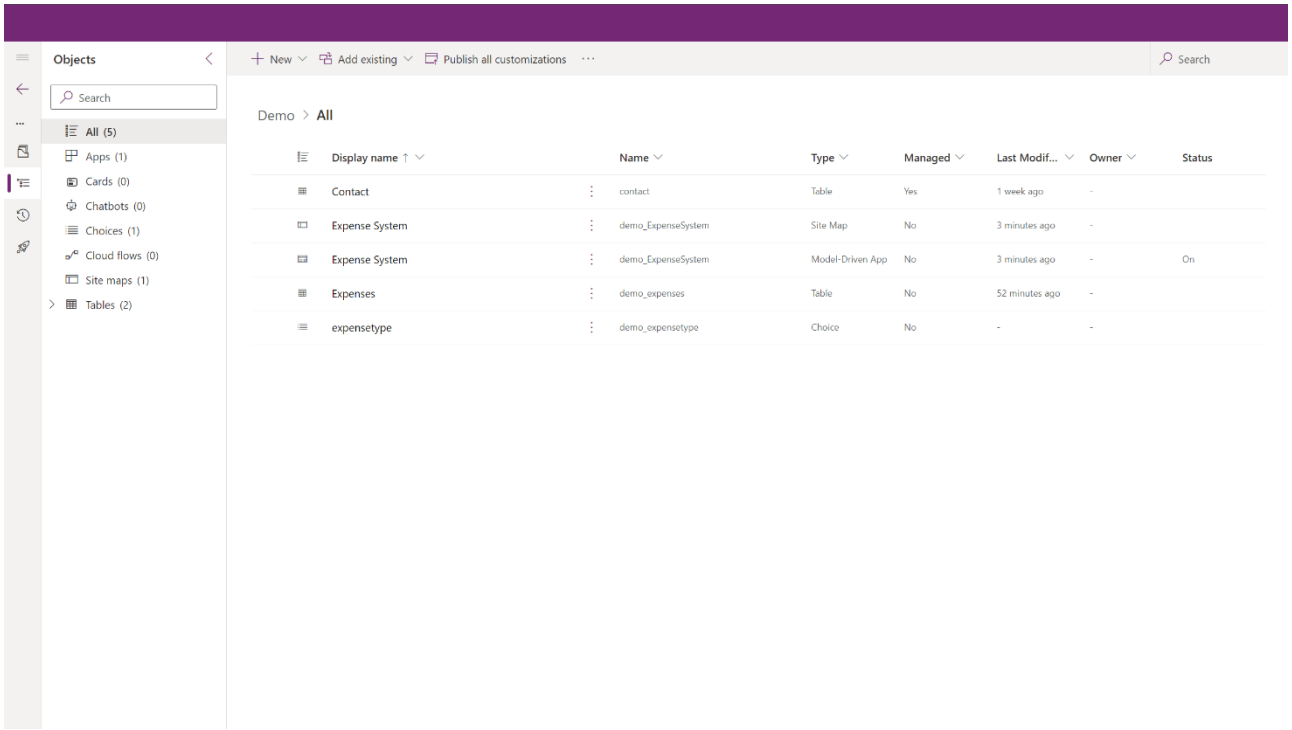
At this point we add a table by clicking on + New, at the right of Pages, and choose Dataverse table. We choose Contact, then Save, Publish and Play.



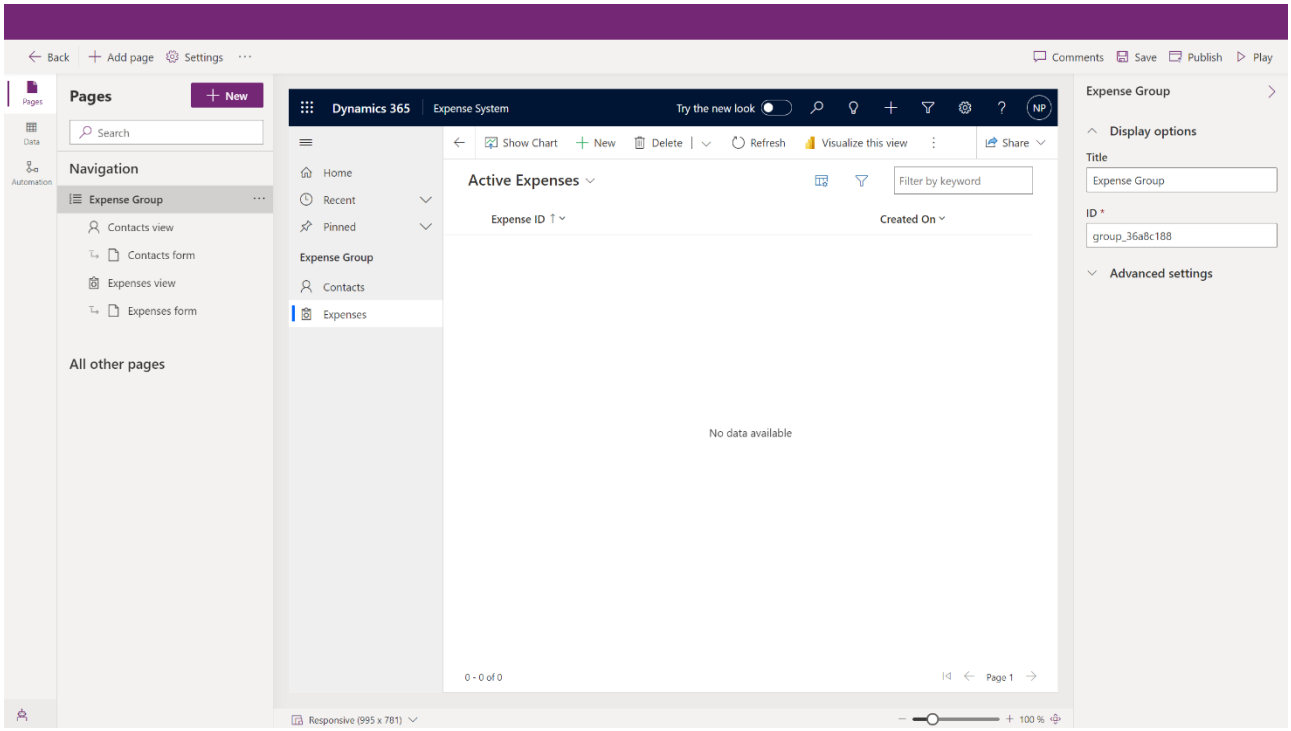
A new Dynamics 365 page will pop up, right into the newly created model-driven app Expense System.



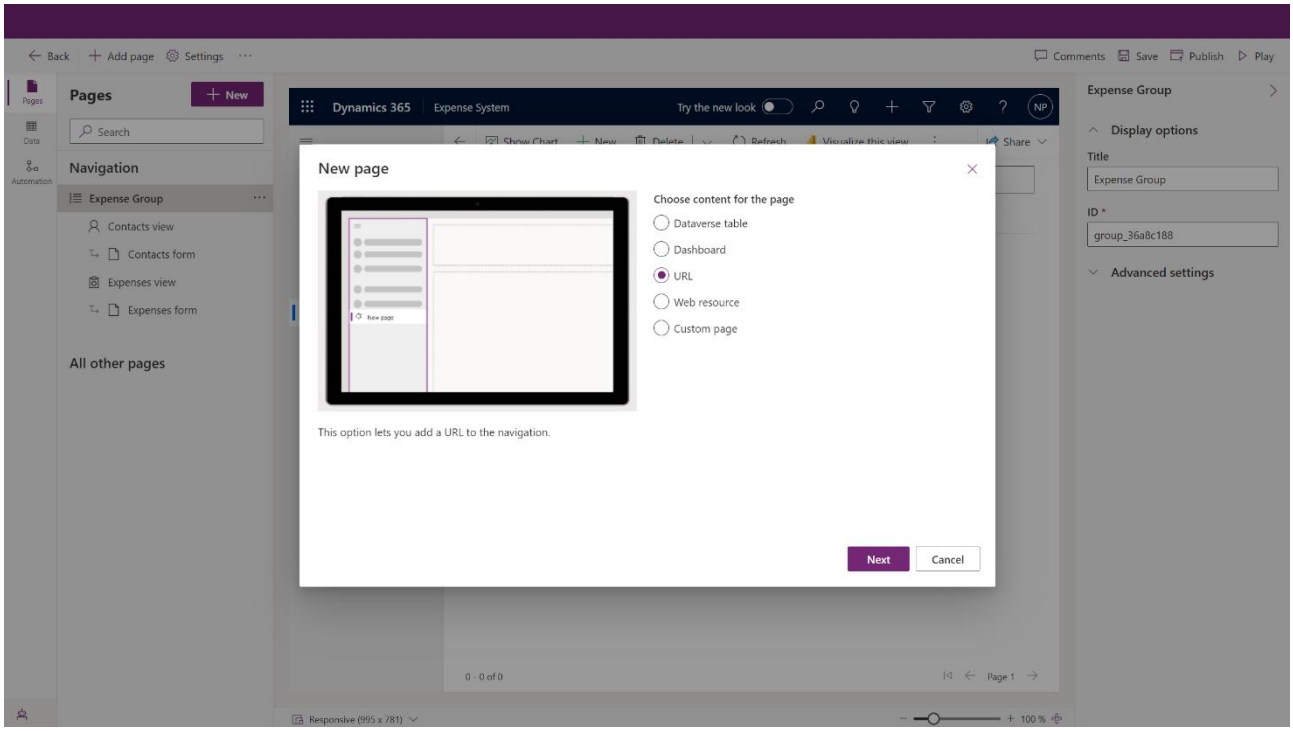
We go back to Power Apps and, when in the Demo > All section, click on Publish all customizations.



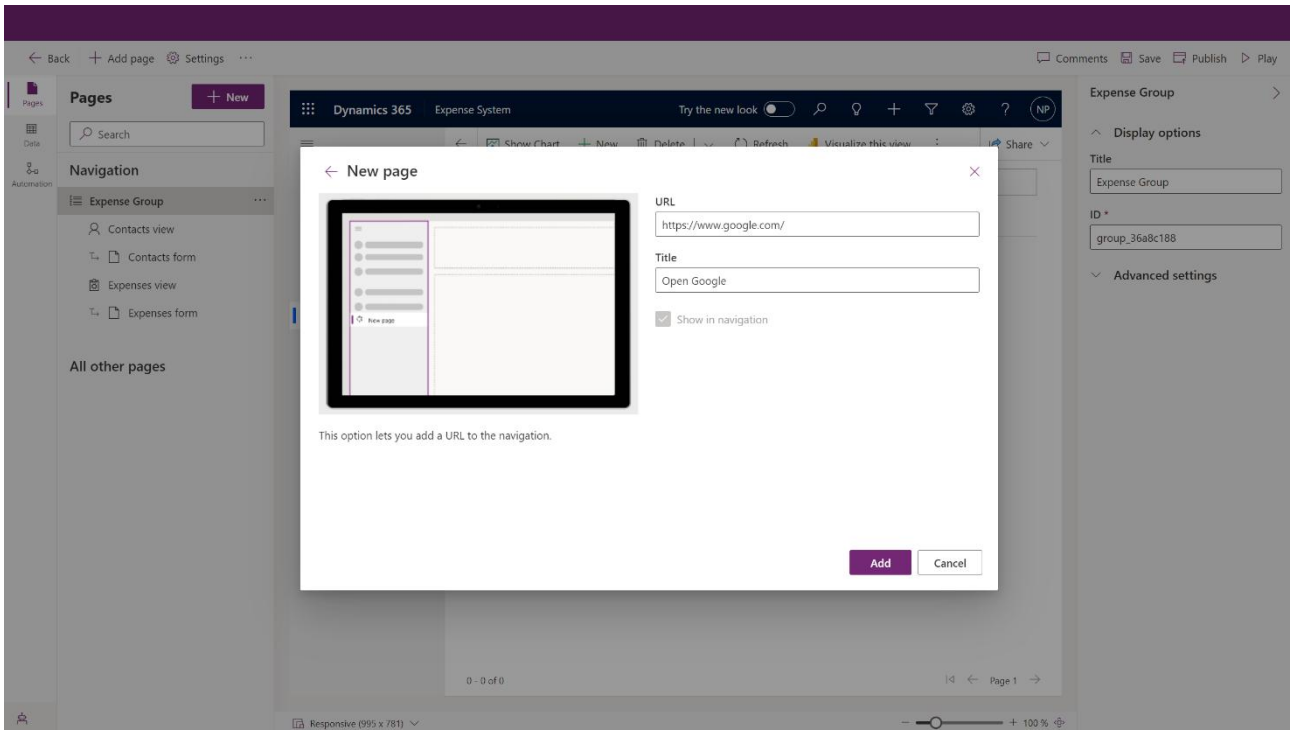
Similarly as before, we add to our Expense Group also the Expenses Dataverse table.



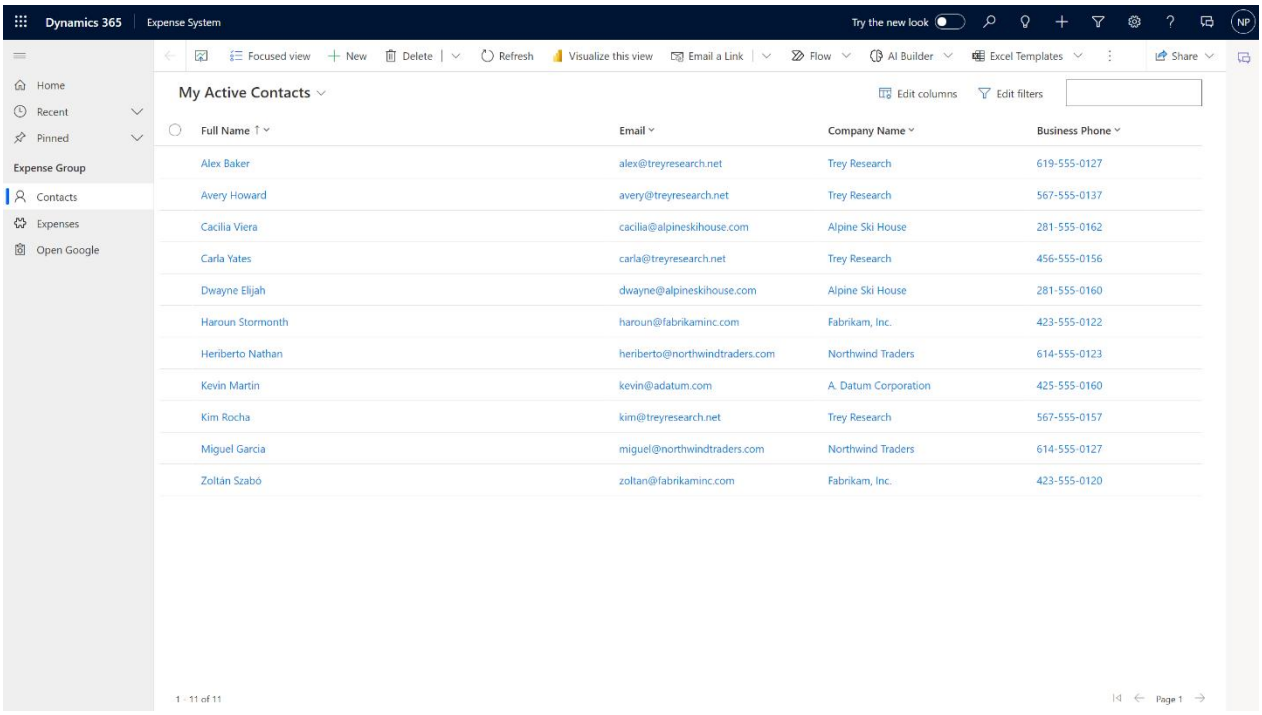
Another thing we can do is to add a URL. This time we add a new page and choose URL.



Say that we want to open Google.



Then click on Add, Save, Publish and Play.



The other options for a new page are Dashboard, Web resource (HTML and JavaScript resources mostly) and Custom page. The first is a visual display of data, typically with charts and graphs, providing an overview of key metrics and insights in a Power Apps environment. The second is an embedded component that allows us to include external web pages or content within our Power Apps environment using an iframe (essentially a window into another web page, providing a way to display content from a different source on the same page.). The third is a blank canvas in Power Apps where we have complete design flexibility to build a custom user interface. It can include controls, components, and even integrated canvas apps.